

Project Title

“Insurance Claim Fraud Detection – AI-Powered Monitoring & Alert System”

Industry

Insurance / FinTech / Risk & Compliance

Target Users

- Insurance Policy Holders
 - Claims Processing Teams
 - Fraud Detection & Compliance Officers
 - Insurance Agents & Brokers
 - Regulatory Authorities
-

Problem Statement

Insurance companies face increasing losses due to **fraudulent claims** such as staged accidents, falsified medical reports, inflated damages, and duplicate claims.

- Manual verification is slow and inefficient.
 - Fraudulent claims often go unnoticed until after payouts, causing financial loss.
 - Lack of real-time fraud detection increases risk exposure.
 - Compliance officers need accurate data to ensure **regulatory adherence**.
-

Proposed Salesforce-Based Solution

A **Salesforce AI-powered fraud detection CRM** that analyzes claims, detects anomalies, and alerts fraud investigators in real time.

Features:

1. Claim Lifecycle Management

- Custom objects for **Policy**, **Claim**, and **Customer Profile**.
- Track claims from submission → review → approval/rejection.

2. AI-Powered Fraud Detection (Einstein AI/ML)

- Detect unusual claim patterns (e.g., same claimant multiple times, mismatched documents, exaggerated expenses).
- Assign a **fraud risk score** to each claim.
- Prioritize suspicious claims for manual review.

3. Document Verification

- Integrate OCR/AI tools to scan uploaded invoices, medical reports, repair bills.
- Flag forged or duplicate documents.

4. Fraud Alerts & Case Creation

- Automatic alert when claim exceeds threshold risk score.
- Salesforce **Case Management** auto-creates an investigation case.
- Assigns fraud officer for resolution.

5. Customer & Agent Profiling

- Track agent performance and claim submission patterns.
- Identify suspicious agents or policyholders with repeat fraudulent activity.

6. Dashboards & Reporting

- Fraud Detection Rate (%)
- Claims at High Risk vs. Low Risk
- Losses Prevented through Fraud Alerts
- Agent/Region-wise Fraud Trends

Outcome

- Reduced financial losses from fraudulent claims.
- Faster claim verification with AI support.
- Improved compliance with insurance regulations.
- Enhanced **trust and transparency** between insurers and customers.

PROJECT TITLE : “Insurance Claim Fraud Detection – AI-Powered Monitoring & Alert System”

Phase 1: Problem Understanding & Industry Analysis

Goal: Understand the insurance industry’s pain points in fraud detection and define clear requirements for the system.

Requirement Gathering

Talk to stakeholders (insurance claim officers, policyholders, fraud investigators, compliance teams).

Example Requirements:

- Capture **customer & policy details** in Salesforce.
 - Log **insurance claims** with supporting documents.
 - Use AI to assign a **fraud risk score** for each claim.
 - Automatically create a **fraud investigation case** if risk score exceeds threshold.
 - Send alerts to claim officers & compliance teams.
 - Generate **fraud detection reports** (losses prevented, high-risk claims, regional patterns).
-

Stakeholder Analysis

- **Admin (You)** → Manage system setup, workflows, AI model integration.
 - **Claim Officers** → Enter and validate claims, review fraud alerts.
 - **Fraud Investigators** → Handle flagged claims, resolve fraud cases.
 - **Compliance Team** → Ensure adherence to regulatory requirements.
 - **Insurance Managers** → Monitor fraud trends, approve high-value claims, analyze dashboards.
 - **Policyholders** → Submit claims via portal, upload documents.
-

Business Process Mapping

Flow of how the system works:

1. Policyholder submits a claim with documents.
 2. Claim Officer logs/validates claim in Salesforce.
 3. AI model scans data → assigns **Fraud Risk Score**.
 - If **Low Risk** → claim continues normal approval process.
 - If **High Risk** → fraud alert triggered → Case created → assigned to Fraud Investigator.
 4. Fraud Investigator reviews evidence & resolves case.
 5. Compliance team monitors flagged claims.
 6. Manager views fraud prevention metrics via dashboards.
-

Industry-Specific Use Case Analysis

In the insurance industry:

- Fraud is a **major loss factor** (false claims, staged accidents, fake medical bills).
- Manual detection is **slow, error-prone, and costly**.
- AI can detect patterns (e.g., multiple claims from same customer, mismatched documents).
- Need **real-time alerts** to prevent fraudulent payouts.
- Compliance is crucial → all fraud cases must be auditable.

So, we need to:

- **Automate fraud detection with AI.**
 - **Track claims lifecycle & investigations in Salesforce.**
 - **Provide managers with dashboards** for insights and risk analysis.
-

AppExchange Exploration

Look for **Insurance & Fraud Detection apps**:

- “Insurance Policy & Claims Management” (some apps exist but are broad).
 - “AI Risk & Fraud Detection” add-ons.
- However, most are **complex enterprise-grade tools**.

We'll build a **simpler custom solution** focused on **fraud detection & case management**, to learn Salesforce + AI integration effectively.

Phase 2: Org Setup & Configuration

1. Salesforce Edition

- Used **Developer Edition Org** for project setup.

2. Company Profile Setup

- Company Name: *Insurance Fraud Detection POC*
- Primary Contact: Email
- Default Locale: *English (India)*
- Default Time Zone: *Asia/Kolkata (GMT+05:30)*
- Default Currency: *INR*

The screenshot shows the 'Company Information' setup page in Salesforce. The page title is 'Insurance Fraud Detection POC'. Below the title, there's a link to 'Help for this Page'. The page content is divided into two main sections: 'Organization Detail' and 'System Information'. The 'Organization Detail' section includes fields for Organization Name, Primary Contact, Division, Address, Fiscal Year Starts In, Activate Multiple Currencies, Enable Data Translation, Newsletter, Admin Newsletter, Hide Notices About System Maintenance, Hide Notices About System Downtime, and Locale Formats. The 'System Information' section includes fields for Phone, Fax, Default Locale, Default Language, Default Time Zone, Currency Locale, Used Data Space, Used File Space, API Requests, Last 24 Hours, Streaming API Events, Last 24 Hours, Restricted Logins, Current Month, Salesforce.com Organization ID, Organization Edition, and Instance. The page also shows a 'Created By' field with the value 'OrgFarm.EPIC, 7/17/2025, 10:46 PM' and a 'Modified By' field with the value 'Sanjivani.Deshmukh, 9/16/2025, 9:32 PM'.

Organization Detail		System Information	
Organization Name	Insurance Fraud Detection POC	Phone	
Primary Contact	sanjivandeshmukh07@gmail.com	Fax	
Division		Default Locale	English (India)
Address	India	Default Language	English
Fiscal Year Starts In	January	Default Time Zone	(GMT+05:30) India Standard Time (Asia/Kolkata)
Activate Multiple Currencies	☐	Currency Locale	English (India) - INR
Enable Data Translation	☐	Used Data Space	442 KB (9%) View
Newsletter	☒	Used File Space	17 KB (0%) View
Admin Newsletter	☒	API Requests, Last 24 Hours	0 (15,000 max)
Hide Notices About System Maintenance	☐	Streaming API Events, Last 24 Hours	0 (10,000 max)
Hide Notices About System Downtime	☐	Restricted Logins, Current Month	0 (0 max)
Locale Formats	ICU	Salesforce.com Organization ID	00DgK000007cSX7
		Organization Edition	Developer Edition
		Instance	CAN96

Created By: OrgFarm.EPIC, 7/17/2025, 10:46 PM
Modified By: Sanjivani.Deshmukh, 9/16/2025, 9:32 PM

3. Business Hours & Holidays

- **Name:** *Claims & Fraud Operations*
- **Time Zone:** *Asia/Kolkata (GMT+05:30)*
- **Default:** ☒
- **Active:** ☒
- **Business Hours:** *Monday – Friday, 9:00 AM – 6:00 PM*
- Leave Sat/Sun unchecked.

SETUP

Business Hours

Organization Business Hours

Help for this Page

Select the days and hours that your support team is available. These hours, when associated with escalation rules, determine the times at which cases can escalate.

If you enter blank business hours for a day, that means your organization does not operate on that day.

Holidays (12)

Business Hours Detail

Edit

Business Hours Name	Claims & Fraud Operations	Time Zone
Business Hours	Sunday 24 Hours Monday 9:00 AM to 6:00 PM Tuesday 9:00 AM to 6:00 PM Wednesday 9:00 AM to 6:00 PM Thursday 9:00 AM to 6:00 PM Friday 9:00 AM to 6:00 PM Saturday 24 Hours	Default Business Hours (GMT+05:30) India Standard Time (Asia/Kolkata)

Active

✓

Created By

Sanjivani Deshmukh 9/15/2025, 4:01 AM

Last Modified By

Sanjivani Deshmukh 9/15/2025, 4:02 AM

Edit

Add common public holidays (example for India 2025):

- **Republic Day** – 26 Jan 2025
- **Independence Day** – 15 Aug 2025
- **Gandhi Jayanti** – 2 Oct 2025
- **Diwali** – 21 Oct 2025
- **Christmas** – 25 Dec 2025
- Check **All-Day**.
- Associated each holiday with your *Claims & Fraud Operations* business hours.

Note: This way, Salesforce knows that during these holidays, escalation rules, case clocks, and workflows pause, so fraud investigation SLAs don't count non-working days.

Edit

Holidays		
Holiday Name	Description	Date and Time
Christmas		12/25/2025 All Day
Diwali		10/21/2025 All Day
Eid al-Fitr		3/31/2025 All Day
Gandhi Jayanti		10/2/2025 All Day
Ganesh Chaturthi		7/27/2025 All Day
Good Friday		4/18/2025 All Day
Holi		3/14/2025 All Day
Independence Day		7/15/2025 All Day
Maha Shivratri		2/26/2025 All Day
Ram Navami		3/30/2025 All Day
Republic Day		1/26/2025 All Day

Back To Top

Always show me more records per related list

4. Fiscal Year Settings

1. Used Standard Fiscal Year (Jan–Dec).

Object Manager

Fiscal Year

Setup

Organization Fiscal Year Edit: Insurance Fraud Detection POC

Help for this Page

To specify the fiscal year type for your organization, choose one of the options below.

Standard Fiscal Year

Custom Fiscal Year

Fiscal Year Information

Your organization can change the fiscal year start month, and specify whether the fiscal year name is set to the starting or ending year. For example, if your fiscal year starts in April 2025 and ends in March 2026, your Fiscal Year setting can be either 2025 or 2026.

Changing the fiscal year shifts fiscal periods and impacts opportunities and forecasts across your organization. If your forecast periods are set to quarterly, adjusting the fiscal year start month will erase existing forecast adjustments and quotas. Consider exporting a data backup before implementing this change.

Change Fiscal Year Period

Save

Cancel

Name

Insurance Fraud Detection POC

Fiscal Year Start Month

January

Fiscal Year is Based On

The ending month

The starting month

Save

Cancel

5. User Setup & Licenses

Created Users: Fraud Manager, Fraud Analyst, Claims Adjuster

SETUP

Users

User

Arjun Manager

User Profile Help for this Page

Permission Set Assignments

Activation Required

Permission Set Group Assignments

Permission Set License Assignments

Personal Groups

Public Group Membership

Queue Membership

Team

Managers in the Role Hierarchy

OAuth Apps

Third-Party Account Links

Built-in Authenticators

Installed Mobile Apps

Authentication Settings for External Systems

Login History

User Provisioning Accounts

User Detail

Edit

Sharing

Reset Password

Login

Freeze

View Summary

Name	Arjun Manager	Role	Fraud Manager
Alias	arju	User License	Salesforce
Email	arjunmanager07@gmail.com	Profile	Fraud Manager Profile
Username	arjunmanager07@gmail.com	Active	☒
Nickname	User17579559917886148747	Marketing User	☐
Title		Offline User	☐
Company		Knowledge User	☐
Department		Flow User	☐
Division		Service Cloud User	☐
Address		Site.com Contributor User	☐
Time Zone	(GMT+05:30) India Standard Time (Asia/Kolkata)	Site.com Publisher User	☐
Locale	English (India)	WDC User	☐
Language	English	Mobile Push Registrations	View
Delegated Approver		Data.com User Type	
Manager		Accessibility Mode (Classic Only)	☐
Receive Approval Request Emails	Only if I am an approver	Debug Mode	☐
Federation ID		High-Contrast Palette on Charts	☐

SETUP

Users

User

Priya Analyst

User Profile Help for this Page

Permission Set Assignments

Activation Required

Permission Set Group Assignments

Permission Set License Assignments

Personal Groups

Public Group Membership

Queue Membership

Team

Managers in the Role Hierarchy

OAuth Apps

Third-Party Account Links

Built-in Authenticators

Installed Mobile Apps

Authentication Settings for External Systems

Login History

User Provisioning Accounts

User Detail

Edit

Sharing

Reset Password

Login

Freeze

View Summary

Name	Priya Analyst	Role	Fraud Analyst
Alias	priy	User License	Salesforce
Email	apriuser@gmail.com	Profile	Fraud Analyst Profile
Username	priya07@gmail.com	Active	☒
Nickname	User17579561657689833711	Marketing User	☐
Title		Offline User	☐
Company		Knowledge User	☐
Department		Flow User	☐
Division		Service Cloud User	☐
Address		Site.com Contributor User	☐
Time Zone	(GMT+05:30) India Standard Time (Asia/Kolkata)	Site.com Publisher User	☐
Locale	English (India)	WDC User	☐
Language	English	Mobile Push Registrations	View
Delegated Approver		Data.com User Type	
Manager		Accessibility Mode (Classic Only)	☐
Receive Approval Request Emails	Only if I am an approver	Debug Mode	☐
Federation ID		High-Contrast Palette on Charts	☐

SETUP

Users

User

Ramesh Adjuster

User Profile Help for this Page

Permission Set Assignments (0) | Permission Set Assignments: Activation Required (0) | Permission Set Group Assignments (0) | Permission Set License Assignments (0) | Personal Groups (0) | Public Group Membership (0) | Queue Membership (0) | Team (0) | Managers in the Role Hierarchy (1) | OAuth Apps (0) | Third-Party Account Links (0) | Built-in Authenticators (0) | Installed Mobile Apps (0) | Authentication Settings for External Systems (0) | Login History (0) | User Provisioning Accounts (0)

User Detail

Edit | Sharing | Reset Password | Login | Freeze | View Summary

Name	Ramesh Adjuster	Role	Claims Adjuster
Alias	rame	User License	Salesforce Platform
Email	rameshadjuster@gmail.com [Verify]	Profile	Standard Platform User
Username	ramesh.adjuster@fraud.com.dev	Active	☒
Nickname	radj	Marketing User	☐
Title		Offline User	☐
Company		Knowledge User	☐
Department		Flow User	☐
Division		Service Cloud User	☐
Address		Site.com Contributor User	☐
Time Zone	(GMT+05:30) India Standard Time (Asia/Kolkata)	Site.com Publisher User	☐
Locale	English (India)	WDC User	☐
Language	English	Mobile Push Registrations	View
Delegated Approver		Data.com User Type	
Manager		Accessibility Mode (Classic Only)	☐
Receive Approval Request Emails	Only if I am an approver	Debug Mode	☐
Federation ID		High-Contrast Palette on Charts	☐

6. Profiles

- Fraud Manager Profile: Full system access.

SETUP

Profiles

Profile

Fraud Manager Profile

Help for this Page

Login IP Ranges (0) | Enabled Apex Class Access (0) | Enabled Visualforce Page Access (0) | Enabled External Data Source Access (0) | Enabled Named Credential Access (0) | Enabled External Credential Principal Access (0) | Enabled Custom Metadata Type Access (0) | Enabled Custom Setting Definitions Access (0) | Enabled Flow Access (0) | Enabled Service Presence Status Access (0) | Enabled Custom Permissions (0)

Profile Detail

Edit | Clone | Delete | View Users

Name	Fraud Manager Profile	Custom Profile	☒
User License	Salesforce		
Description			
Created By	Sanjivani Dashmukh: 9/16/2025, 7:44 AM	Modified By	Sanjivani Dashmukh: 9/16/2025, 7:45 AM

Page Layouts

Standard Object Layouts	
Global	Global Layout [View Assignment]
Email Application	Not Assigned [View Assignment]
Home Page Layout	Home Page Default [View Assignment]
Account	Account Layout [View Assignment]
Alternative Payment Method	Alternative Payment Method Layout [View Assignment]
Location Group	Location Group Layout [View Assignment]
Location Group Assignment	Location Group Assignment Layout [View Assignment]
Macro	Macro Layout [View Assignment]
Object Milestone	Object Milestone Layout [View Assignment]
Operating Hours	Operating Hours Layout [View Assignment]

- **Fraud Analyst Profile:** Access to fraud analysis objects.

SETUP Profiles

Profile: **Fraud Analyst Profile**

Users with this profile have the permissions and page layouts listed below. Administrators can change a user's profile by editing that user's personal information.

If your organization uses Record Types, use the Edit links in the Record Type Settings section below to make one or more record types available to users with this profile.

[Login IP Ranges \(0\)](#) |
 [Enabled Apex Class Access \(0\)](#) |
 [Enabled Visualforce Page Access \(0\)](#) |
 [Enabled External Data Source Access \(0\)](#) |
 [Enabled Named Credential Access \(0\)](#) |
 [Enabled External Credential Principal Access \(0\)](#) |
 [Enabled Custom Metadata Type Access \(0\)](#) |
 [Enabled Custom Settings Definitions Access \(0\)](#) |
 [Enabled Flow Access \(0\)](#) |
 [Enabled Service Presence Status Access \(0\)](#) |
 [Enabled Custom Permissions \(0\)](#)

Profile Detail [Edit](#) [Clone](#) [Delete](#) [View Users](#)

Name	Fraud Analyst Profile		
User License	Salesforce	Custom Profile	✓
Description			
Created By	Sanjivani Deshmukh, 9/16/2025, 5:37 AM	Modified By	Sanjivani Deshmukh, 9/16/2025, 7:42 AM

Page Layouts

Standard Object Layouts	Global	Global Layout View Assignment	Location Group	Location Group Layout View Assignment
Email Application	Not Assigned	View Assignment	Location Group Assignment	Location Group Assignment Layout View Assignment
Home Page Layout	Home Page Default	View Assignment	Macro	Macro Layout View Assignment

- **Claims Adjuster Profile:** Create & edit claims only.

SETUP Profiles

Profile: **Claims Adjuster Profile**

Users with this profile have the permissions and page layouts listed below. Administrators can change a user's profile by editing that user's personal information.

If your organization uses Record Types, use the Edit links in the Record Type Settings section below to make one or more record types available to users with this profile.

[Login IP Ranges \(1\)](#) |
 [Enabled Apex Class Access \(0\)](#) |
 [Enabled Visualforce Page Access \(0\)](#) |
 [Enabled External Data Source Access \(0\)](#) |
 [Enabled Named Credential Access \(0\)](#) |
 [Enabled External Credential Principal Access \(0\)](#) |
 [Enabled Custom Metadata Type Access \(0\)](#) |
 [Enabled Custom Settings Definitions Access \(0\)](#) |
 [Enabled Flow Access \(0\)](#) |
 [Enabled Service Presence Status Access \(0\)](#) |
 [Enabled Custom Permissions \(0\)](#)

Profile Detail [Edit](#) [Clone](#) [Delete](#) [View Users](#)

Name	Claims Adjuster Profile		
User License	Salesforce	Custom Profile	✓
Description			
Created By	Sanjivani Deshmukh, 9/16/2025, 1:27 AM	Modified By	Sanjivani Deshmukh, 9/16/2025, 7:44 AM

Page Layouts

Standard Object Layouts	Global	Global Layout View Assignment	Location Group	Location Group Layout View Assignment
Email Application	Not Assigned	View Assignment	Location Group Assignment	Location Group Assignment Layout View Assignment
Home Page Layout	Home Page Default	View Assignment	Macro	Macro Layout View Assignment
Account	Account Layout		Object Milestone	Object Milestones Layout

7. Roles

- **Fraud Manager (top)** → full visibility.
- **Fraud Analysts** → below manager.
- **Adjusters** → below manager.

SETUP Roles

Creating the Role Hierarchy

You can build on the existing role hierarchy shown on this page. To insert a new role, click **Add Role**.

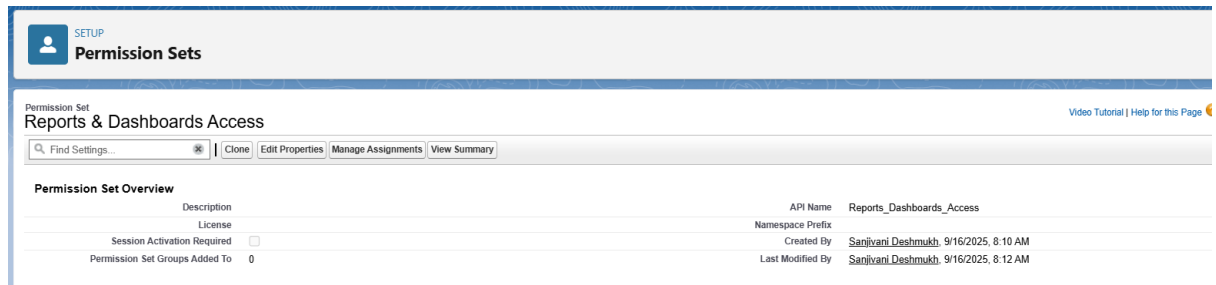
Your Organization's Role Hierarchy

[Collapse All](#) [Expand All](#)

- Insurance Fraud Detection POC
 - Add Role
 - CEO [Edit](#) [Del](#) [Assign](#)
 - Add Role
 - Fraud Manager [Edit](#) [Del](#) [Assign](#)
 - Add Role
 - Claims Adjuster [Edit](#) [Del](#) [Assign](#)
 - Add Role
 - Fraud Analyst [Edit](#) [Del](#) [Assign](#)
 - Add Role

8. Permission Sets

- For Fraud Reporting Access : Extra report/dashboard access.



SETUP
Permission Sets

Permission Set
Reports & Dashboards Access

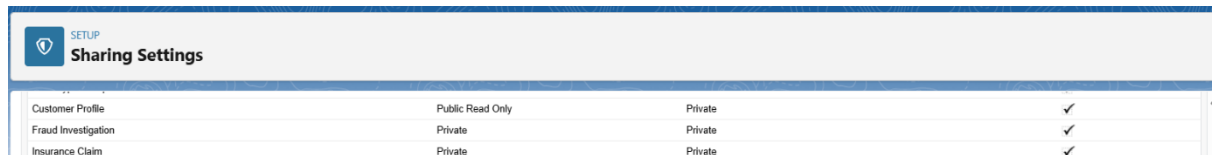
Find Settings... Clone Edit Properties Manage Assignments View Summary

Permission Set Overview

Description	API Name	Reports_Dashboards_Access
License	Namespace Prefix	
Session Activation Required ☐	Created By	Sanjivani Deshmukh 9/16/2025, 8:10 AM
Permission Set Groups Added To 0	Last Modified By	Sanjivani Deshmukh 9/16/2025, 8:12 AM

9. OWD (Org-Wide Defaults)

- Insurance Claim → Private (owner & manager only).
- Customer Profile → Public Read Only.
- Fraud Investigation → Private.

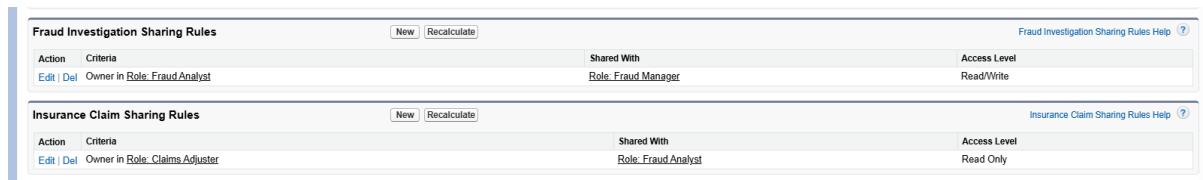


SETUP
Sharing Settings

Customer Profile	Public Read Only	Private	✓
Fraud Investigation	Private	Private	✓
Insurance Claim	Private	Private	✓

10. Sharing Rules

- Added rules for **Analysts** to view claims created by Adjusters.



Fraud Investigation Sharing Rules

New Recalculate

Fraud Investigation Sharing Rules Help

Action	Criteria	Shared With	Access Level
Edit Del	Owner In Role: Fraud Analyst	Role: Fraud Manager	Read/Write

Insurance Claim Sharing Rules

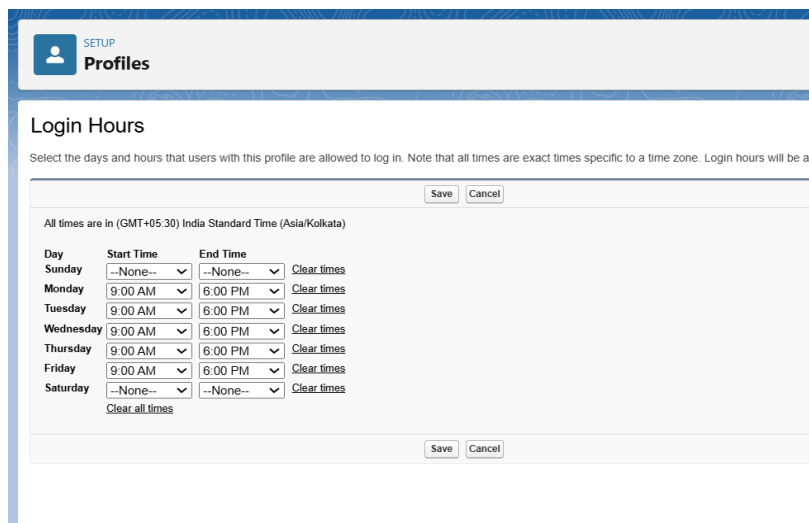
New Recalculate

Insurance Claim Sharing Rules Help

Action	Criteria	Shared With	Access Level
Edit Del	Owner In Role: Claims Adjuster	Role: Fraud Analyst	Read Only

11. Login Access Policies

- Restricted working hours login (9 AM – 6 PM) for Adjusters.



SETUP
Profiles

Login Hours

Select the days and hours that users with this profile are allowed to log in. Note that all times are exact times specific to a time zone. Login hours will be app

Save Cancel

All times are in (GMT+05:30) India Standard Time (Asia/Kolkata)

Day	Start Time	End Time	Clear times
Sunday	--None--	--None--	Clear times
Monday	9:00 AM	6:00 PM	Clear times
Tuesday	9:00 AM	6:00 PM	Clear times
Wednesday	9:00 AM	6:00 PM	Clear times
Thursday	9:00 AM	6:00 PM	Clear times
Friday	9:00 AM	6:00 PM	Clear times
Saturday	--None--	--None--	Clear times

Clear all times

Save Cancel

12. Dev Org Setup

- Used **Developer Org as Sandbox** for building and testing.

13. Sandbox Usage

- If this were a real company, we'd build in Sandbox, then deploy to Production.

14. Deployment Basics

- This project has no Production, as no Sandbox used.

PROJECT TITLE : “Insurance Claim Fraud Detection – AI-Powered Monitoring & Alert System”

Phase 3: Data Modelling & Relationships

1. Standard & Custom Objects

- **Standard Objects we can leverage:**
 - User (Adjuster, Analyst, Manager)
 - Account (could represent Policy Holder or Customer if needed)
- **Custom Objects:**
 - Insurance Claim (tracks claim details)

SETUP > OBJECT MANAGER
Insurance Claim

Details

Fields & Relationships
Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
Record Types
Related Lookup Filters
Restriction Rules
Scoping Rules
Object Access

Details

Description

API Name
Insurance_Claim__c

Custom
✓

Singular Label
Insurance Claim

Plural Label
Insurance Claims

Enable Reports
✓

Track Activities
✓

Track Field History

Deployment Status
Deployed

Help Settings
Standard salesforce.com Help Window

Edit Delete

- Customer Profile (personal & policy details)

SETUP > OBJECT MANAGER
Customer Profile

Details

Fields & Relationships
Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
Record Types
Related Lookup Filters
Restriction Rules
Scoping Rules
Object Access

Details

Description

API Name
Customer_Profile__c

Custom
✓

Singular Label
Customer Profile

Plural Label
Customer Profiles

Enable Reports
✓

Track Activities
✓

Track Field History

Deployment Status
Deployed

Help Settings
Standard salesforce.com Help Window

Edit Delete

- Fraud Investigation (fraud case records)

SETUP > OBJECT MANAGER
Fraud Investigation

Details
Fields & Relationships
Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
Record Types
Related Lookup Filters
Restriction Rules
Scoping Rules
Object Access

Details

Edit Delete

Description

API Name
Fraud_Investigation__c

Custom
✓

Singular Label
Fraud Investigation

Plural Label
Fraud Investigations

Enable Reports
✓

Track Activities
✓

Track Field History

Deployment Status
Deployed

Help Settings
Standard salesforce.com Help Window

2. Fields:

Insurance Claim (Insurance_Claim__c)

- Claim Number → Auto Number (CLM-{0000}) (already set as Record Name)
- Policy Number → Text (20)
- Claim Amount → Currency (16,2)
- Claim Type → Picklist {Accident, Theft, Medical, Other}
- Claim Date → Date
- Status → Picklist {Submitted, In Review, Approved, Rejected, Flagged}
- Risk Score → Number (0–100)

SETUP > OBJECT MANAGER
Insurance Claim

Details	Fields & Relationships 12 Items. Sorted by Field Label		
Fields & Relationships	FIELD LABEL	FIELD NAME	DATA TYPE
Page Layouts	Claim Amount	Claim_Amount__c	Currency(18, 0)
Lightning Record Pages	Claim Date	Claim_Date__c	Date
Buttons, Links, and Actions	Claim Number	Claim_Number__c	Auto Number
Compact Layouts	Claim Type	Claim_Type__c	Picklist
Field Sets	Created By	CreatedById	Lookup(User)
Object Limits	Customer Profile	Customer_Profile__c	Lookup(Customer Profile)
Record Types	Insurance Claim Name	Name	Text(80)
Related Lookup Filters	Last Modified By	LastModifiedById	Lookup(User)
Restriction Rules	Owner	OwnerId	Lookup(User:Group)
Scoping Rules	Policy Number	Policy_Number__c	Text(20)
Object Access	Risk Score	Risk_Score__c	Number(3, 0)
Triggers	Status	Status__c	Picklist
Flow Triggers			
Validation Rules			
Conditional Field Formatting			

Customer Profile (Customer_Profile__c)

- Customer Name → Text (Record Name)
- Date of Birth → Date
- Email → Email
- Phone → Phone
- Address → Text Area (255)
- Policy Number → Text (20) (must match Claims Policy Number)
- Occupation → Picklist {Salaried, Self-Employed, Business Owner, Retired, Student, Unemployed}

SETUP > OBJECT MANAGER

Customer Profile

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Restriction Rules

Scoping Rules

Object Access

Triggers

Flow Triggers

Validation Rules

Fields & Relationships

11 Items, Sorted by Field Label

FIELD LABEL	FIELD NAME	DATA TYPE
Address	Address__c	Text Area(255)
Created By	CreatedById	Lookup(User)
Customer Name	Customer_Name__c	Text(20)
Customer Profile Name	Name	Text(80)
Date of Birth	Date_of_Birth__c	Date
Email	Email__c	Email
Last Modified By	LastModifiedById	Lookup(User)
Occupation	Occupation__c	Picklist
Owner	OwnerId	Lookup(User,Group)
Phone	Phone__c	Phone
Policy Number	Policy_Number__c	Text(20)

Fraud Investigation (Fraud_Investigation__c)

- Investigation ID → Auto Number (FRA-{0000}) (Record Name)
- Related Claim → Lookup (Insurance Claim)
- Fraud Score → Number (0–100)
- Analyst Notes → Long Text Area (32768, 5 visible lines)
- Investigation Status → Picklist {Open, In Progress, Closed}

SETUP > OBJECT MANAGER

Fraud Investigation

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Restriction Rules

Scoping Rules

Object Access

Triggers

Flow Triggers

Fields & Relationships

10 Items, Sorted by Field Label

FIELD LABEL

FIELD NAME

DATA TYPE

Analyst Notes

Analyst_Notes__c

Long Text Area(32768)

Created By

CreatedById

Lookup(User)

Fraud Investigation Name

Name

Text(80)

Fraud Score

Fraud_Score__c

Number(3, 0)

Insurance Claim

Insurance_Claim__c

Lookup(Insurance Claim)

Investigation ID

Investigation_ID__c

Auto Number

Investigation Status

Investigation_Status__c

Picklist

Last Modified By

LastModifiedById

Lookup(User)

Owner

OwnerId

Lookup(User,Group)

Related Claim

Related_Claim__c

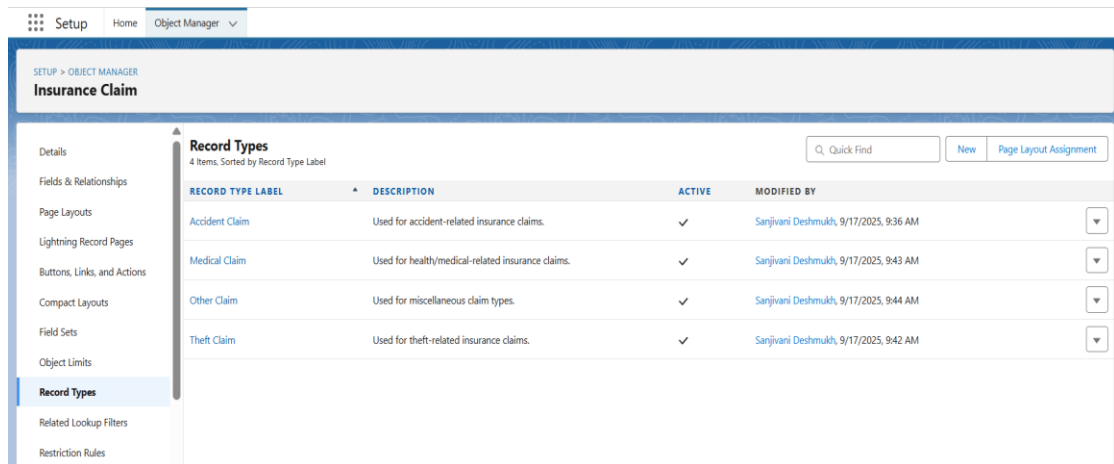
Lookup(Insurance Claim)

3. Record Types:

Insurance Claim — Record Types

Multiple record types based on claim nature:

- Accident Claim
- Theft Claim
- Medical Claim
- Other Claim



4. Page Layouts:

- **Insurance Claim Layouts**

Accident Claim Layout

- Sections:
 - General Claim Info: Claim Number, Claim Type, Policy Number, Claim Amount, Status
 - Accident Information: Claim Date, Accident Details (custom field), Risk Score
 - System Info: Created By, Last Modified By, Owner

Medical Claim Layout

- Sections:
 - General Claim Info: Claim Number, Claim Type, Policy Number, Claim Amount, Status, Claim Date
 - Medical Information: Hospital Name (custom), Treatment Cost (custom), Medical Notes (custom)
 - Risk Assessment: Risk Score
 - System Info: Created By, Last Modified By, Owner

Theft Claim Layout

- Sections:
 - General Claim Info: Claim Number, Claim Type, Policy Number, Claim Amount, Status, Claim Date
 - Theft Information: Police Report Number (custom), Stolen Item Details (custom)
 - Risk Assessment: Risk Score
 - System Info: Created By, Last Modified By, Owner

Other Claim Layout

- Sections:
 - General Claim Info: Claim Number, Claim Type, Policy Number, Claim Amount, Status, Claim Date
 - Additional Notes: Generic Notes field
 - Risk Assessment: Risk Score
 - System Info: Created By, Last Modified By, Owner

SETUP > OBJECT MANAGER
Insurance Claim

Details
Fields & Relationships
Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits
Record Types
Related Lookup Filters
Restriction Rules
Scoping Rules

Page Layouts
5 Items, Sorted by Page Layout Name

Q, Quick Find New Page Layout Assignment

PAGE LAYOUT NAME	CREATED BY	MODIFIED BY	
Accident Claim Layout	Sanjivani Deshmukh, 9/17/2025, 9:53 AM	Sanjivani Deshmukh, 9/17/2025, 10:14 AM	▼
Insurance Claim Layout	Sanjivani Deshmukh, 9/16/2025, 1:34 AM	Sanjivani Deshmukh, 9/17/2025, 10:14 AM	▼
Medical Claim Layout	Sanjivani Deshmukh, 9/17/2025, 10:10 AM	Sanjivani Deshmukh, 9/17/2025, 10:20 AM	▼
Other Claim Layout	Sanjivani Deshmukh, 9/17/2025, 10:34 AM	Sanjivani Deshmukh, 9/17/2025, 10:34 AM	▼
Theft Claim Layout	Sanjivani Deshmukh, 9/17/2025, 10:09 AM	Sanjivani Deshmukh, 9/17/2025, 10:14 AM	▼

Fraud Investigation Layout (Using Default one)

- General Info: Investigation ID, Related Claim, Fraud Score, Analyst Notes, Investigation Status
- System Info: Created By, Last Modified By, Owner

SETUP > OBJECT MANAGER
Fraud Investigation

Details
Fields & Relationships
Page Layouts
Lightning Record Pages
Buttons, Links, and Actions
Compact Layouts
Field Sets
Object Limits

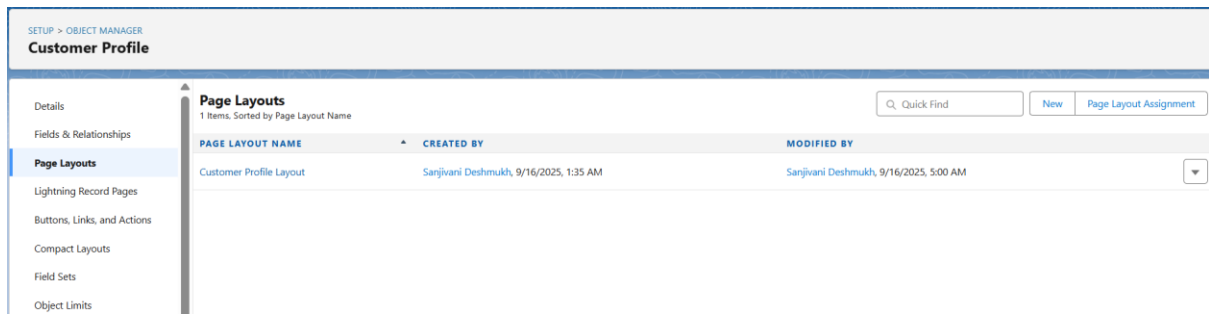
Page Layouts
1 Items, Sorted by Page Layout Name

Q, Quick Find New Page Layout Assignment

PAGE LAYOUT NAME	CREATED BY	MODIFIED BY	
Fraud Investigation Layout	Sanjivani Deshmukh, 9/16/2025, 1:35 AM	Sanjivani Deshmukh, 9/16/2025, 5:01 AM	▼

Customer Profile Layout (Using Default one)

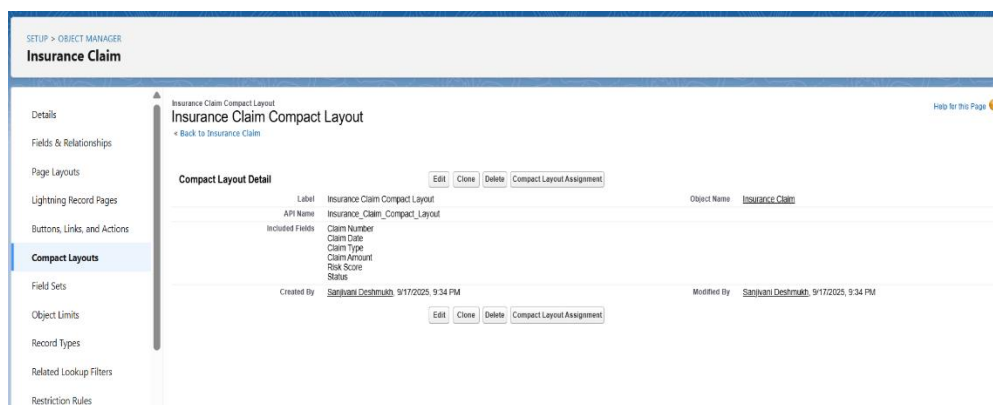
- Customer Details: Customer Name, Date of Birth, Email, Phone, Address, Occupation, Policy Number
- System Info: Created By, Last Modified By, Owner



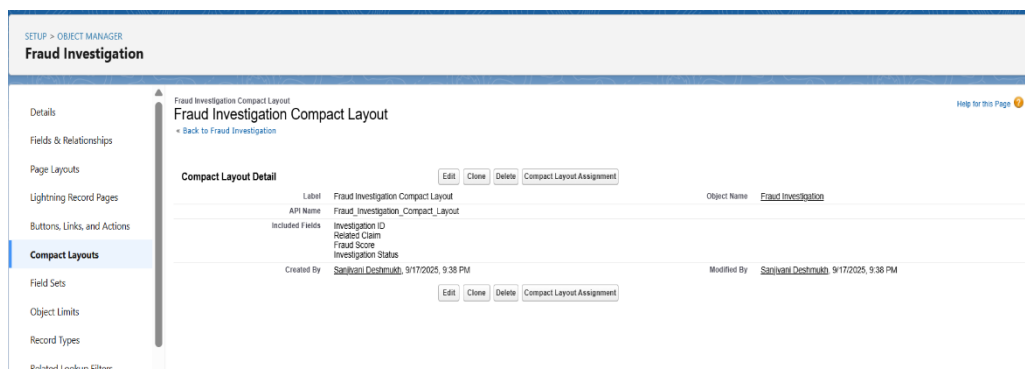
5. Compact Layouts:

This ensures that users see the most critical record information at a glance in the highlight panel.

- **Insurance Claim Compact Layout**
 - Claim Number
 - Claim Type
 - Claim Amount
 - Status
 - Risk Score

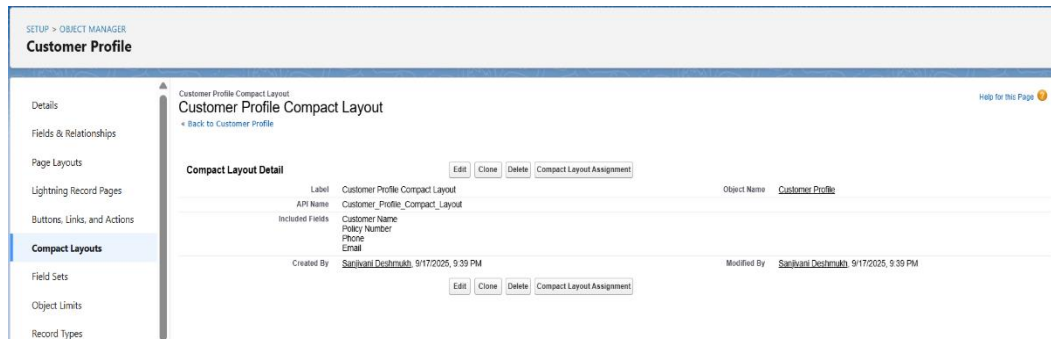


- **Fraud Investigation Compact Layout**
 - Investigation ID
 - Related Claim
 - Fraud Score
 - Investigation Status



- **Customer Profile Compact Layout**

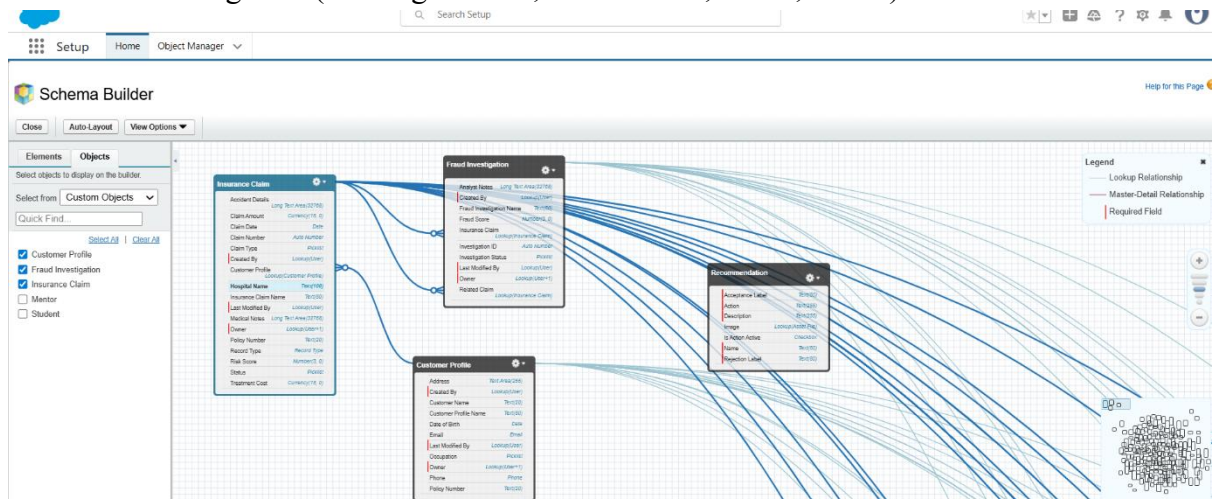
- Customer Name
- Policy Number
- Phone
- Email



6. Schema Builder

Schema View

- Customer Profile (Policy Number, Name, DOB, Email, Phone, etc.)
 ↓ Lookup →
- Insurance Claim (Claim Number, Claim Type, Claim Amount, Claim Date, Status, Risk Score, Accident/Medical/Theft Details)
 ↓ Lookup →
- Fraud Investigation (Investigation ID, Fraud Score, Notes, Status)



7. Junction Objects:

No junction objects created, junction objects could be added for advanced scenarios like if we want to track **multiple Customers linked to a single Claim** or if we want to link a **single Investigation to multiple Claim**.

8. External Objects:

No external Objects created, could be added for real-time fraud scoring by connecting Salesforce with external transaction data sources.

TITLE: “Insurance Claim Fraud Detection – AI-Powered Monitoring & Alert System”

Phase 4: Process Automation (Admin)

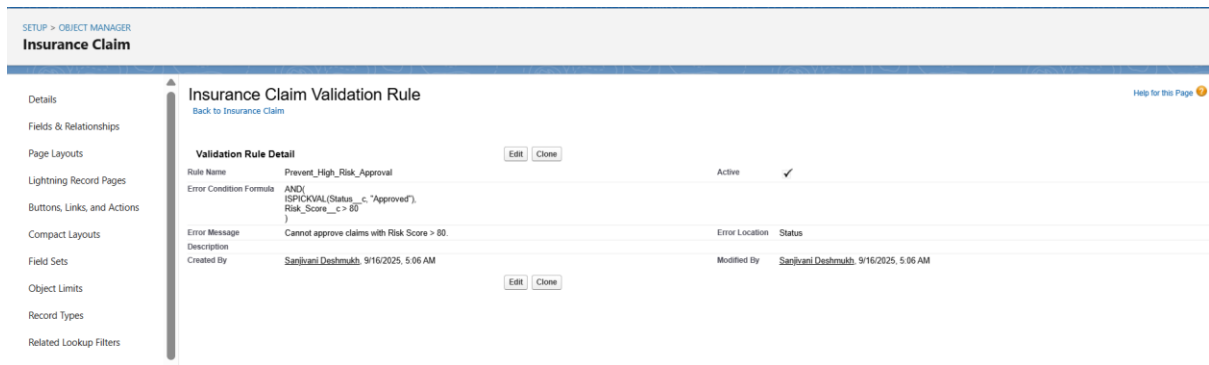
Goal: Automate claim-handling, fraud detection and escalation so suspicious claims are flagged, routed and tracked with minimal manual effort.

1. Validation Rules:

- **Purpose:** Enforce data quality and prevent risky actions (e.g., approving a high-risk claim). These run at save-time and block invalid states.
- **High_Claim_Risk_Score** — prevent saving large claims with low risk assessment.
 - **Logic:** if Claim_Amount__c > 500000 AND Risk_Score__c < 80 then block.
 - Formula: AND(
Claim_Amount__c > 500000,
Risk_Score__c < 80
)
 - **Error message:** “High claim amount requires Risk Score ≥ 80.” (placed on Risk Score)

The screenshot shows the Salesforce Setup page for the 'Insurance Claim' object. The left sidebar contains navigation links: Details, Fields & Relationships, Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, and Object Limits. The main content area is titled 'Insurance Claim Validation Rule' with a 'back to Insurance Claim' link. Below the title is a 'Validation Rule Detail' section with 'Edit' and 'Clone' buttons. The rule is named 'High_Claim_Risk' and is active. The error condition formula is 'Claim_Amount__c > 500000 && Risk_Score__c < 70'. The error message is 'Claims above \$5,00,000 must have a Risk Score ≥ 70'. The error location is 'Risk Score'. The rule was created by 'Sanjivani Deshmukh' on 9/16/2025 at 5:04 AM and modified by the same user at the same time. There are 'Edit' and 'Clone' buttons at the bottom of the rule details.

- **Approved_High_Risk_Not_Allowed** — prevent approving a claim with very high risk.
 - **Logic:** if Status = Approved AND Risk Score > 80 then block.
 - Formula: AND(
ISPICKVAL(Status__c, "Approved"),
Risk_Score__c > 80
)
 - **Error message:** “Claims with Risk Score > 80 cannot be approved.” (placed on Status)



2. Workflow Rules (legacy):

- **Purpose:** Workflow Rules in Salesforce help automate immediate actions when certain conditions are met (like sending alerts, updating fields, or assigning tasks).
- **Example of Workflow Rule**

High-Risk Claim Notification (Implemented in Flows)

- **Object:** Insurance Claim
- **Rule Criteria:** Risk Score ≥ 70
- **Action:**
 - Send **Email Alert** to the Fraud Analyst.
 - Create a **Task** for the Adjuster: *"Review High-Risk Claim."*

3. Process Builder (legacy):

- **Purpose:** Process Builder allows multiple conditions, multiple actions, and record updates across objects.
- **Example of Process Builder**

Auto-Update Claim Status on High Fraud Score (Implemented in Flows)

- **Trigger:** When Fraud Investigation is created/updated.
- **Condition:** Fraud Score ≥ 70 .
- **Action:** Update related Insurance Claim $\rightarrow$ Status = *"In Review"*.

4. Approval Process:

- **Purpose:** Require manager approval for high-value claims (compliance & control). Locks the record until final decision.
- **Approval process name: High Value Claim Approval**
 - **Entry criteria:** Claim_Amount__c > 500000

- **Approver:** Fraud Manager (can be a specific user or manager role)
- **Initial submission action:** lock record, send submission email to submitter
- **Final approval actions:**
- **Field Update** — **Claim Status = Approved** (Field Update name: Set_Status_Approved)
- **Email Alert** — **Approval Confirmation to Customer** (template: *Approval Confirmation*)
- **Final rejection actions:**
- **Field Update** — **Claim Status = Rejected** (Field Update name: Set_Status_Rejected)
- **Email Alert** — **Notify Analyst of Rejection**

Approval Processes

Insurance Claim: High Value Claim Approval

Process Definition Detail

Process Name	High Value Claim Approval	Active	☐
Unique Name	High_Value_Claim_Approval	Next Automated Approver Determined By	
Description			
Entry Criteria	Insurance Claim: Claim Amount GREATER THAN 500000		
Record Editability	Administrator ONLY	Allow Submitters to Recall Approval Requests	☐
Approval Assignment Email Template			
Initial Submitters	Insurance Claim Owner		
Created By	Sanjivani Dashmukh, 9/18/2025, 8:33 AM		
Modified By	Sanjivani Dashmukh, 9/18/2025, 8:38 AM		

Initial Submission Actions

Action Type	Description
Record Lock	Lock the record from being edited

Approval Steps

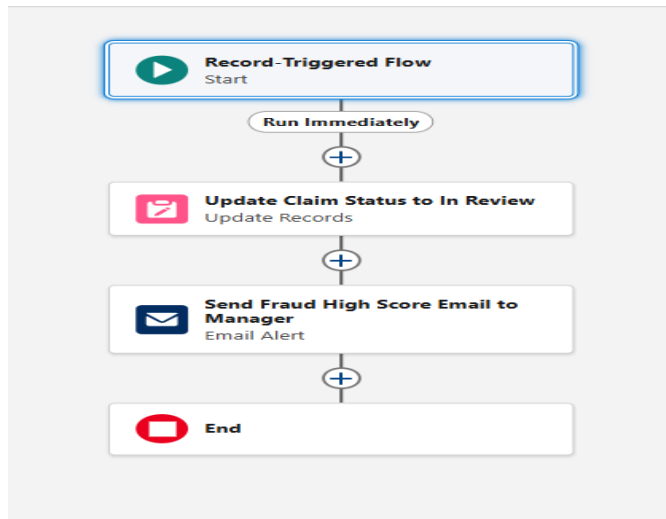
Action	Step Number	Name	Description	Criteria	Assigned Approver	Reject Behavior
Step 1	1	Step 1			User Arjun Manager	Final Rejection

5. Flow Builder:

- **Purpose:** All active automation is implemented in Flow Builder. Created Record-Triggered Flows and a Scheduled-Triggered Flow.

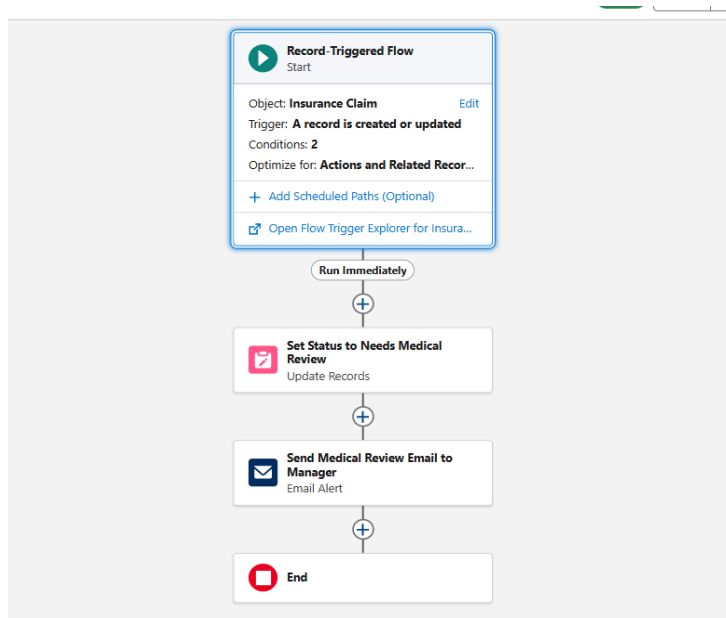
Flow A — Fraud_Score_AutoFlag_Flow (Record-Triggered Flow)

- **Object:** Fraud_Investigation
- **Trigger:** When record is created or updated
- **Entry condition:** `Fraud_Score__c >= 70`
- **Optimize:** Actions and Related Records
- **Immediate actions:**
 - **Update Records** → find **Invoice Claim** where `Id = $Record.Related_Claim__c` → set `Status__c = "In Review"`.
 - **Action (Email Alert)** → send **Fraud High Score Alert** to Manager.



Flow B — Medical_Claim_Review_Flow (Record-Triggered Flow)

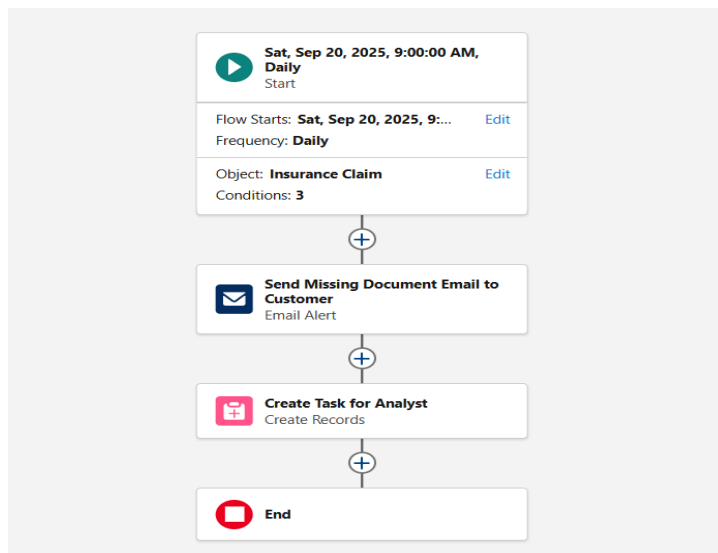
- **Object:** Insurance_Claim
- **Trigger:** When record is created or updated
- **Entry condition:** ISPICKVAL(Claim_Type__c,"Medical") AND Claim_Amount__c > 200000
- **Immediate actions:**
 1. **Update Records** → use the record that triggered the flow → set Status__c = "Needs Medical Review".
 2. **Action (Email Alert)** → send Medical Review Alert to Manager.



Flow C — Missing_Documents_Reminder_Flow (Scheduled-Triggered Flow)

- **Schedule:** Daily at 9:00 AM
- **Object:** Insurance_Claim

- **Condition:** ISPICKVAL(Status__c,"Submitted") AND ISPICKVAL(Document_Status__c,"Pending") AND CreatedDate <= (TODAY() - 3)
- **Actions per matching record:**
 1. **Action (Email Alert)** → Missing Document Reminder (recipient: Customer email from related Customer Profile).
 2. **Create Records** → Task: Subject = “Follow up on missing claim documents”, OwnerId = Analyst user or Analyst Queue, ActivityDate = TODAY() + 1, WhatId = claim Id.



6. Email Alerts

- **Purpose:** Reusable email-alert records (with templates) are easiest to call from Flows/Approval processes.

Email alerts created (examples and recipients)

- **Fraud High Score Alert to Manager** → object: Fraud Investigation → template: *Fraud High Score Alert* → recipients: Fraud Manager (User or Role).

The screenshot shows the 'Email Alerts' setup page. The title is 'Email Alerts'. Below it, the specific alert is 'Fraud High Score Alert to Manager'. There are links for 'Edit', 'Delete', and 'Clone'. The 'Email Alert Detail' section shows the following information:

Description	Fraud High Score Alert to Manager	Email Template	Sales_New Customer Email
Unique Name	Fraud_High_Score_Alert_to_Manager	Object	Fraud Investigation
From Email Address	Current User's email address		
Recipients	User_Arjun Manager		
Additional Emails			
Created By	Sanjivani Deshmukh, 9/18/2025, 8:56 AM	Modified By	Sanjivani Deshmukh, 9/18/2025, 8:56 AM

At the bottom, there are links for 'Edit', 'Delete', and 'Clone'.

- **Medical Review Alert to Manager** → object: Insurance Claim → template: *Medical Claim Needs Review* → recipient: Manager.

SETUP

Email Alerts

Email Alert

Medical Review Alert to Manager

[Rules Using This Email Alert \(0\)](#) |
 [Approval Processes Using This Email Alert \(0\)](#) |
 [Entitlement Processes Using This Email Alert \(0\)](#)

Email Alert Detail

Description

Medical Review Alert to Manager

Email Template

Sales: New Customer Email

Unique Name

Medical_Review_Alert_to_Manager

Object

Insurance Claim

From Email Address

Current User's email address

Recipients

User: Arjun Manager

Additional Emails

Created By

Sanjivani Deshmukh, 9/20/2025, 9:39 AM

Modified By

Sanjivani Deshmukh, 9/20/2025, 9:39 AM

- Missing Document Reminder** → object: Insurance Claim → template: *Please upload missing documents* → recipient: related Customer email (Customer_Profile__r.Email__c) or direct Contact lookup.

SETUP

Email Alerts

Email Alert

Missing Document Reminder

[Rules Using This Email Alert \(0\)](#) |
 [Approval Processes Using This Email Alert \(0\)](#) |
 [Entitlement Processes Using This Email Alert \(0\)](#)

Email Alert Detail

Description

Missing Document Reminder

Email Template

Sales: New Customer Email

Unique Name

Missing_Document_Reminder

Object

Insurance Claim

From Email Address

Current User's email address

Recipients

Related User: Last Modified By

Additional Emails

Created By

Sanjivani Deshmukh, 9/20/2025, 9:49 AM

Modified By

Sanjivani Deshmukh, 9/20/2025, 9:49 AM

7.Field Updates:

Purpose: Automatic field updates are performed inside Flows or as Approval Process field update actions.

Examples implemented

- In Flows:** Update Status__c to **In Review** or **Needs Medical Review** (Flow elements).
- In Approval Process:** Final Approval action sets Status__c = Approved; Final Rejection sets Status__c = Rejected.

Object Manager

Field Updates

All Workflow Field Updates

[Help for this Page](#)

Field updates allow you to automatically change a field value to one that you specify. Field updates are actions associated with workflow rules and approval processes.

View:

All Workflow Field Updates

[Edit](#) |
 [Create New View](#)

Action

Name ↑

Field to Update

Operation

Value

Last Modified Date

[Edit](#) |
 [Del](#)

[Set Status Approved](#)

Insurance Claim: Status

Value

Approved

9/18/2025

[Edit](#) |
 [Del](#)

[Set Status Rejected](#)

Insurance Claim: Status

Value

Rejected

9/18/2025

[Edit](#) |
 [Del](#)

[Set Status Medical Review](#)

Insurance Claim: Status

Value

In Review

9/18/2025

8. Tasks:

Purpose: Create follow-up work items for analysts and adjusters when human action is needed.

Examples implemented

- **Scheduled Flow** → create Task for Analyst: Subject “Follow up on missing claim documents”, Related To = claim, Owner = Analyst user or Queue, ActivityDate = TODAY() + 1.

SETUP

Tasks

Task

Follow up on Pending Documents

Rules Using This Task (1) | Approval Processes Using This Task (0) | Entitlement Processes Using This Task (0)

Help for this Page

Workflow Task Detail

Edit | Delete | Clone

Object	Insurance Claim	Status	Not Started
Assigned To	User: Priya Analyst	Priority	Normal
Subject	Follow up on Pending Documents		
Unique Name	Analyst_Document_Followup		
Due Date	Rule Trigger Date + 1 days		
Comments			
Created By	Sanjivani Deshmukh, 9/18/2025, 3:10 AM	Modified By	Sanjivani Deshmukh, 9/18/2025, 3:10 AM

Edit | Delete | Clone

Rules Using This Task

Rules Using This Task Help

Action	Rule Name	Description	Object	Active
Edit Del Deactivate	Missing_Document_Followup		Insurance Claim	✓

TITLE: “Insurance Claim Fraud Detection – AI-Powered Monitoring & Alert System”

Phase 5: Apex Programming (Developer)

Goal: Implement server-side logic to support complex/frequent operations that cannot be handled (or are inefficient) with clicks alone — bulk-safe, test-covered, and maintainable code for fraud detection and claim processing.

1. Classes & Objects:

Purpose

Contain business logic in reusable, testable units (Apex classes) and keep triggers thin.

Implementation

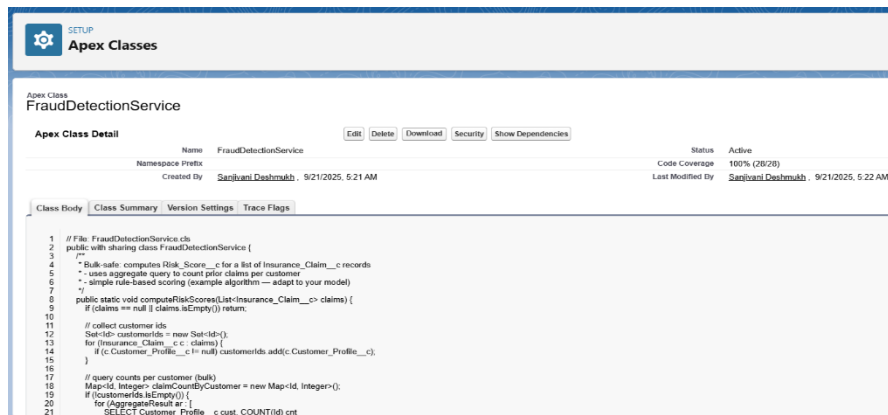
- **FraudDetectionService** — core business logic to calculate **Risk_Score__c** for **Insurance_Claim__c** records in bulk.
- Small utility classes where needed (e.g., email helpers, constants).

Why

- **Encapsulation:** single place to change scoring logic.
- **Reusability:** service can be called from triggers, batches, schedulers, tests.
- **Testability:** easy to unit test pure logic.

Implementation steps

1. Create a new Apex Class in Developer Console (File → New → Apex Class) named **FraudDetectionService**.
2. Implement a bulk-safe public static method like **computeRiskScores(List<Insurance_Claim__c> claims)** that updates **Risk_Score__c** in-memory.
3. Keep no DML inside loops; use aggregate queries to fetch supporting data.



2. Apex Triggers (before/after insert/update/delete)

Purpose

React to DML events to run server logic before or after records are saved.

Implementation

- **InsuranceClaimTrigger** (before insert, before update, after insert, after update) — delegates to handler.
 - Before: call **FraudDetectionService.computeRiskScores()** to set **Risk_Score__c**.
 - After insert/update: create **Fraud_Investigation__c** if claim is flagged/high-risk.
- **FraudInvestigationTrigger** (after insert, after update) — delegates to handler to update related claim(s) status and enqueue notifications.

Why

- before triggers update fields on the same record prior to DML (efficient).
- after triggers create related records (need Ids) and perform cross-object updates.

Implementation steps

1. Create trigger file (Developer Console → New → Apex Trigger). only delegate to a handler.
2. Create a handler class **InsuranceClaimTriggerHandler** and **FraudInvestigationHandler** with public static methods for each event.
3. Ensure bulk-safe coding: pass List<...> into handler methods and process collections.

SETUP

Apex Triggers

Apex Trigger

FraudInvestigationTrigger

Apex Trigger Detail

Edit

Delete

Download

Show Dependencies

Name	FraudInvestigationTrigger	sObject Type	Fraud Investigation
Code Coverage	100% (2/2)	Status	Active
Created By	Sanjivani Deshmukh	Last Modified By	Sanjivani Deshmukh
Created	9/21/2025, 5:26 AM	Last Modified	9/21/2025, 5:31 AM
Namespace Prefix			

Apex TriggerVersion SettingsTrace Flags

```
1 // File: FraudInvestigationTrigger.trigger
2 trigger FraudInvestigationTrigger on Fraud_Investigation__c (after insert, after update) {
3     if (Trigger.isAfter) {
4         FraudInvestigationHandler.afterInsertOrUpdate(Trigger.new);
5     }
6 }
```

Edit

Delete

Download

Show Dependencies

SETUP

Apex Triggers

Apex Trigger

InsuranceClaimTrigger

Apex Trigger Detail

Edit

Delete

Download

Show Dependencies

Name	InsuranceClaimTrigger	sObject Type	Insurance Claim
Code Coverage	100% (5/5)	Status	Active
Created By	Sanjivani Deshmukh	Last Modified By	Sanjivani Deshmukh
Created	9/21/2025, 5:23 AM	Last Modified	9/21/2025, 5:25 AM
Namespace Prefix			

Apex TriggerVersion SettingsTrace Flags

```
1 // File: InsuranceClaimTrigger.trigger
2 trigger InsuranceClaimTrigger on Insurance_Claim__c (before insert, before update, after insert, after update) {
3     if (Trigger.isBefore) {
4         if (Trigger.isInsert) InsuranceClaimTriggerHandler.beforeInsert(Trigger.new);
5         if (Trigger.isUpdate) InsuranceClaimTriggerHandler.beforeUpdate(Trigger.new, Trigger.oldMap);
6     }
7     if (Trigger.isAfter) {
8         if (Trigger.isInsert) InsuranceClaimTriggerHandler.afterInsert(Trigger.new);
9         if (Trigger.isUpdate) InsuranceClaimTriggerHandler.afterUpdate(Trigger.new, Trigger.oldMap);
10    }
11 }
```

Edit

Delete

Download

Show Dependencies

SETUP

Apex Classes

Apex Class

FraudInvestigationHandler

Apex Class Detail

Edit

Delete

Download

Security

Show Dependencies

Name	FraudInvestigationHandler	Status	Active
Namespace Prefix		Code Coverage	100% (22/22)
Created By	Sanjivani Deshmukh	Last Modified By	Sanjivani Deshmukh
Created	9/21/2025, 5:27 AM	Last Modified	9/21/2025, 5:31 AM

Class BodyClass SummaryVersion SettingsTrace Flags

```
1 // File: FraudInvestigationHandler.cls
2 public with sharing class FraudInvestigationHandler {
3     public static void afterInsertOrUpdate(List<Fraud_Investigation__c> invList) {
4         Set<Id> claimIds = new Set<Id>();
5         for (Fraud_Investigation__c inv : invList) {
6             if (inv.Related_Claim__c != null) claimIds.add(inv.Related_Claim__c);
7         }
8         if (claimIds.isEmpty()) return;
9
10        // aggregate max fraud score per claim
11        Map<Id, Decimal> maxScoreByClaim = new Map<Id, Decimal>();
12        for (AggregateResult ar : [
13            SELECT Related_Claim__c, MAX(Fraud_Score__c) maxScore
14            FROM Fraud_Investigation__c
15            WHERE Related_Claim__c IN :claimIds
16            GROUP BY Related_Claim__c
17        ]) {
18            Id old = (Id) ar.get('c');
19            Decimal m = (Decimal) ar.get('maxScore');
20            maxScoreByClaim.put(old, m);
21        }
```

SETUP

Apex Classes

Apex Class

InsuranceClaimTriggerHandler

Apex Class Detail

Edit

Delete

Download

Security

Show Dependencies

Name	InsuranceClaimTriggerHandler	Status	Active
Namespace Prefix		Code Coverage	73% (17/23)
Created By	Sanjivani Deshmukh	Last Modified By	Sanjivani Deshmukh
Created	9/21/2025, 5:24 AM	Last Modified	9/21/2025, 5:25 AM

Class BodyClass SummaryVersion SettingsTrace Flags

```
1 // File: InsuranceClaimTriggerHandler.cls
2 public with sharing class InsuranceClaimTriggerHandler {
3     public static void beforeInsert(List<Insurance_Claim__c> newList) {
4         // compute risk for new records
5         FraudDetectionService.computeRiskScores(newList);
6     }
7
8     public static void beforeUpdate(List<Insurance_Claim__c> newList, Map<Id, Insurance_Claim__c> oldMap) {
9         // recompute risk when relevant fields changed
10        FraudDetectionService.computeRiskScores(newList);
11
12        // example safeguard: prevent saving Approved when risk > 80
13        for (Insurance_Claim__c c : newList) {
14            if (c.Status__c == 'Approved' && c.Risk_Score__c != null && c.Risk_Score__c > 80) {
15                c.addError('Claims with Risk Score > 80 cannot be approved');
16            }
17        }
18    }
19
20    public static void afterInsert(List<Insurance_Claim__c> newList) {
21        // create initial fraud investigations for very high-risk claims
```

3. Trigger Design Pattern:

Purpose

Organize trigger logic to be maintainable, testable and avoid duplication.

Implementation

- Thin trigger files delegating to a ***Handler** class. Handler methods are organized by context (**beforeInsert, beforeUpdate, afterInsert, afterUpdate**).

Why

- Keeps triggers small and readable.
- Easier to unit test handler methods.
- Supports reuse and avoids mixed responsibilities.

Implementation notes

- Handler methods accept collections (e.g., **List<Insurance_Claim__c> newList**), and when needed **Map<Id, SObject> oldMap**.
- Add a central Handler class that calls the **FraudDetectionService** or other service classes.

4. SOQL & SOSL:

Purpose

Query data from Salesforce; SOSL for text search across objects.

Implementation

- SOQL aggregated query to count prior claims per customer (used in scoring).
- SOQL to fetch Users (e.g., managers) when sending notifications.
- No heavy SOSL needed in current scope; mention available for future search use-cases.

Why

- Required to compute counts and to find related records (**Fraud_Investigation__c** aggregates or find managers).

Implementation steps

- Use aggregate queries for counts:

- `List<AggregateResult> arList = [SELECT Customer_Profile__c c, COUNT(Id) cnt
FROM Insurance_Claim__c WHERE Customer_Profile__c IN :customerIds GROUP BY
Customer_Profile__c];`
 - Always limit fields and records returned; use **WHERE** clauses to reduce data.

5. Collections: List, Set, Map:

Purpose

Efficiently store and manipulate data in bulk operations.

Used

- **Set<Id>** to collect unique **Customer_Profile__c** ids.
- **Map<Id, Integer>** to store counts per customer.
- **List<Insurance_Claim__c>** for DML operations.

Why

- Prevent duplicates (Set), fast lookup (Map), ordered DML (List).

Implementation notes

- Build sets from trigger records, then query using IN :set.
- Map aggregate results to a map for O(1) lookup.

6. Control Statements:

Purpose

Implement decision logic (if/else, for, while, switch).

Used

- if-else to apply score buckets and to decide when to create investigations.
- for loops to iterate over collections.

Why

- Core to any logic; used for scoring rules and record transformations.

7. Batch Apex:

Purpose

Process large volumes of records asynchronously within governor limits.

Implementation

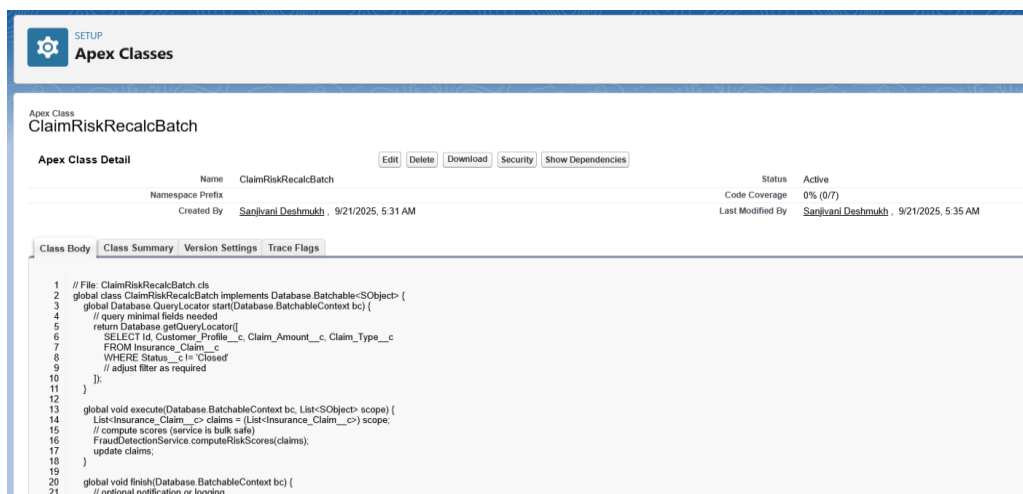
- **ClaimRiskRecalcBatch** — a **Database.Batchable<SObject>** implementation that recalculates risk scores for claims in batches (chunk size e.g., 200).

Why

- If you have hundreds/thousands of claims, batch jobs let you recompute scores without hitting limits.

Implementation steps

1. Create batch class implementing start, execute, finish.
2. start returns a **Database.getQueryLocator()** with the claims to process.
3. execute calls **FraudDetectionService.computeRiskScores()** and updates chunk.
4. Schedule the batch via **System.schedule()** or a scheduled Apex class.



The screenshot displays the Salesforce Apex Classes page for the **ClaimRiskRecalcBatch** class. The interface includes a header with the 'Apex Classes' title and a 'SETUP' button. Below the title, there are tabs for 'Apex Class Detail', 'Class Body', 'Class Summary', 'Version Settings', and 'Trace Flags'. The 'Apex Class Detail' tab is active, showing the class name, namespace prefix, and creation/modification details. The 'Class Body' tab is also visible, showing the Apex code for the class. The code implements the **Database.Batchable<SObject>** interface, with methods for **start**, **execute**, and **finish**. The **execute** method calls **FraudDetectionService.computeRiskScores()** to update claims.

```
1 // File: ClaimRiskRecalcBatch.cls
2 global class ClaimRiskRecalcBatch implements Database.Batchable<SObject> {
3     global Database.QueryLocator start(Database.BatchableContext bc) {
4         // query minimal fields needed
5         return Database.getQueryLocator(
6             SELECT Id, Customer_Profile__c, Claim_Amount__c, Claim_Type__c
7             FROM Insurance_Claim__c
8             WHERE Status__c != 'Closed'
9             // adjust filter as required
10        );
11    }
12
13    global void execute(Database.BatchableContext bc, List<SObject> scope) {
14        List<Insurance_Claim__c> claims = (List<Insurance_Claim__c>) scope;
15        // compute scores (service is bulk safe)
16        FraudDetectionService.computeRiskScores(claims);
17        update claims;
18    }
19
20    global void finish(Database.BatchableContext bc) {
21        // optional notification or logging
22    }
23 }
```

8. Queueable Apex:

Purpose

Asynchronous job for medium complexity tasks (chaining allowed) — lighter than Batch for smaller jobs.

Implementation

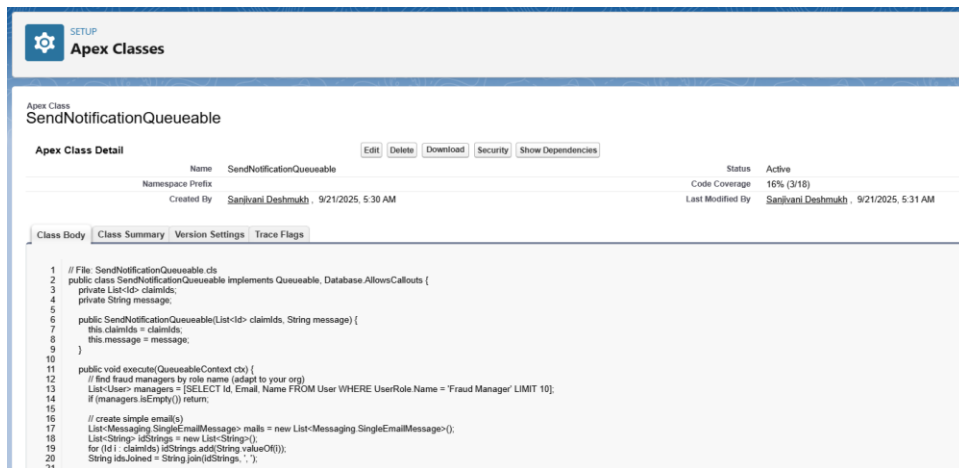
- **SendNotificationQueueable** to send emails or callouts asynchronously after fraud threshold reached.

Why

- Offloads email sending or callouts from trigger context to avoid long-running operations and limits.

Implementation notes

- Implement Queueable and call **System.enqueueJob(new SendNotificationQueueable(ids, message))**.
- If callouts are needed in queueable, implement **Database.AllowsCallouts**.



Apex Class Detail

Name	Status
SendNotificationQueueable	Active

Namespace Prefix: Code Coverage: 16% (3/18)

Created By: Sanjivani Deshmukh, 9/21/2025, 5:30 AM Last Modified By: Sanjivani Deshmukh, 9/21/2025, 5:31 AM

Class Body

```

1 // File: SendNotificationQueueable.cls
2 public class SendNotificationQueueable implements Queueable, Database.AllowsCallouts {
3     private List<Id> claimIds;
4     private String message;
5
6     public SendNotificationQueueable(List<Id> claimIds, String message) {
7         this.claimIds = claimIds;
8         this.message = message;
9     }
10
11     public void execute(QueueableContext ctx) {
12         // find fraud managers by role name (adapt to your org)
13         List<User> managers = [SELECT Id, Email, Name FROM User WHERE UserRole Name = 'Fraud Manager' LIMIT 10];
14         if (managers.isEmpty()) return;
15
16         // create simple email(s)
17         List<Messaging.SingleEmailMessage> mails = new List<Messaging.SingleEmailMessage>();
18         List<String> idStrings = new List<String>();
19         for (Id i : claimIds) idStrings.add(String.valueOf(i));
20         String idJoined = String.join(idStrings, ', ');
21     }

```

9. Scheduled Apex

Purpose

Run jobs on a schedule (daily/nightly).

Implementation

- **ClaimRiskRecalcScheduler** that executes **ClaimRiskRecalcBatch** nightly.

Why

- Regularly keep risk scores up-to-date and detect emerging fraud patterns.

How to schedule

- From Setup → Apex Classes → Schedule Apex, or use:
System.schedule('Nightly Claim Risk', '0 0 2 * * ?', new ClaimRiskRecalcScheduler());

Apex Class
ClaimRiskRecalcScheduler

Apex Class Detail [Edit] [Delete] [Download] [Security] [Show Dependencies]

Name	ClaimRiskRecalcScheduler	Status	Active
Namespace Prefix		Code Coverage	0% (0/3)
Created By	Sanjivani Deshmukh - 9/21/2025, 5:32 AM	Last Modified By	Sanjivani Deshmukh - 9/21/2025, 5:35 AM

Class Body [Class Summary] [Version Settings] [Trace Flags]

```

1 // File: ClaimRiskRecalcScheduler.cls
2 global class ClaimRiskRecalcScheduler implements Schedulable {
3     global void execute(SchedulableContext sc) {
4         ClaimRiskRecalcBatch batch = new ClaimRiskRecalcBatch();
5         Database.executeBatch(batch, 200);
6     }
7 }

```

[Edit] [Delete] [Download] [Security] [Show Dependencies]

10. Future Methods

Purpose

Asynchronous execution for callouts that cannot be done synchronously in triggers.

Implementation

- **ExternalFraudApiCaller.callExternalService(List<Id> claimIds)** with **@future(callout=true)** as a wrapper to call an external fraud API and update scores.

Why

- Trigger context cannot perform callouts; **@future** or **Queueable** allows callouts asynchronously.

Important

- In tests, use **Test.setMock(HttpCalloutMock.class, ...)** to simulate external responses.

Apex Class
ExternalFraudApiCaller

Apex Class Detail [Edit] [Delete] [Download] [Security] [Show Dependencies]

Name	ExternalFraudApiCaller	Status	Active
Namespace Prefix		Code Coverage	95% (23/24)
Created By	Sanjivani Deshmukh - 9/21/2025, 5:37 AM	Last Modified By	Sanjivani Deshmukh - 9/21/2025, 5:39 AM

Class Body [Class Summary] [Version Settings] [Trace Flags]

```

1 // File: ExternalFraudApiCaller.cls
2 public class ExternalFraudApiCaller {
3     @future(callout=true)
4     public static void callExternalService(List<Id> claimIds) {
5         try {
6             List<Insurance_Claim__c> claims = [SELECT Id, Claim_Amount__c, Claim_Type__c FROM Insurance_Claim__c WHERE Id IN :claimIds];
7             List<Insurance_Claim__c> updates = new List<Insurance_Claim__c>();
8             Http http = new Http();
9             for (Insurance_Claim__c c : claims) {
10                 HttpRequest req = new HttpRequest();
11                 req.setEndpoint('https://example-fraud-api.test/score'); // replace with your endpoint
12                 req.setMethod('POST');
13                 req.setHeader('Content-Type', 'application/json');
14                 Map<String, Object> payload = new Map<String, Object>{
15                     'claimId' => c.Id,
16                     'amount' => c.Claim_Amount__c,
17                     'type' => c.Claim_Type__c
18                 };
19                 req.setBody(JSON.serialize(payload));
20                 HttpResponse res = http.send(req);
21                 if (res.getStatusCode() == 200) {

```

11. Exception Handling

Purpose

Catch and handle runtime errors without breaking user transactions unnecessarily.

Implementation

- try-catch blocks in callout and queueable classes; in triggers added **addError()** to block invalid saves when appropriate.

Why

- Provide graceful fallback (e.g., set default score if API fails) and log errors for troubleshooting.

12. Test Classes

Purpose

Unit tests validate logic, ensure $\geq 75\%$ coverage and allow safe deployment.

Implementation

- **TestFraudDetection** test class with multiple test methods:
 - Insert customer & claim → assert **Risk_Score__c** computed and investigation created.
 - Insert **Fraud_Investigation__c** with **high Fraud_Score__c** → assert related claim status becomes In Review.
 - Mock HTTP callout and test **@future** call to external fraud API.

Why

- Required for deployment. Ensures code works across scenarios and handles async jobs.

Implementation notes

- Use **@isTest** annotation.
- Use **Test.startTest()** / **Test.stopTest()** around async operations to force execution within tests.
- Use **Test.setMock(HttpCalloutMock.class, new MockHttpResponseGenerator())** for callouts.

Apex Class

TestFraudDetection

Apex Class Detail

Edit

Delete

Download

Run Test

Show Dependencies

Name	TestFraudDetection	Status	Active
Namespace Prefix		Created By	Sanjivani Deshmukh
Last Modified By	Sanjivani Deshmukh		9/21/2025, 5:39 AM

Class Body

Class Summary

Version Settings

Trace Flags

```

1 // File: TestFraudDetection.cls
2 @isTest
3 private class TestFraudDetection {
4     @isTest static void testComputeRiskAndInvestigationCreation() {
5         // create customer
6         Customer_Profile__c cust = new Customer_Profile__c(Customer_Name__c='T1', Policy_Number__c='POL-001');
7         insert cust;
8
9         // create a claim that should get a high risk score
10        Insurance_Claim__c claim = new Insurance_Claim__c(
11            Customer_Profile__c = cust.id,
12            Claim_Amount__c = 600000,
13            Claim_Type__c = 'Accident',
14            Status__c = 'Submitted'
15        );
16
17        Test.startTest();
18        insert claim;
19        Test.stopTest();
20
21        // claim = ISFI FCT Id Risk Score -> FROM Insurance__c claim -> WHFBD Id = claim Id

```

13. Asynchronous Processing (summary)

Purpose

Use Queueable, Batch, Scheduled Apex and Future to move heavy/remote/blocking work out of immediate transaction.

Used

- Queueable for notifications, Batch for nightly recalculation, Scheduled for running batch, Future for external API callouts.

Why

- Protects user experience, respects governor limits, supports scale.

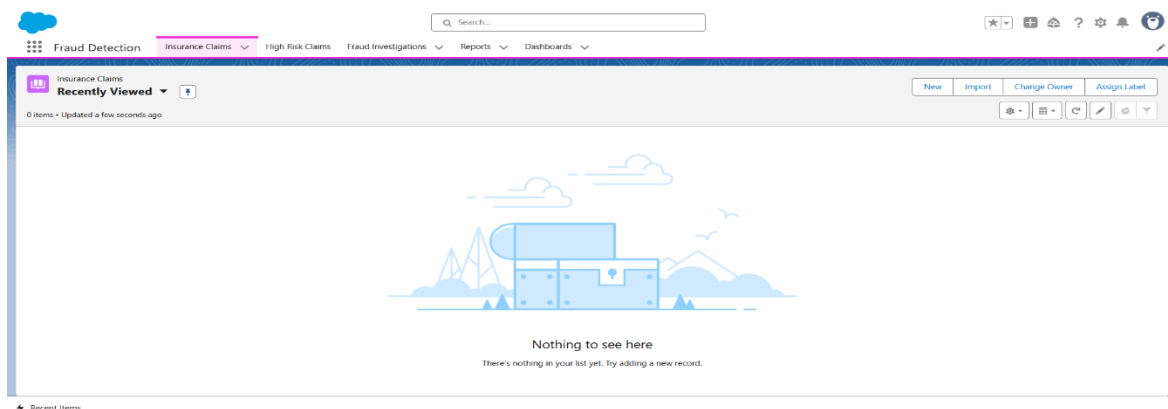
Phase 6: User Interface Development

Goal:

Build a user-friendly and interactive Salesforce Lightning interface for Fraud Detection, enabling Adjusters, Analysts, and Managers to access claims, view fraud scores, and take actions seamlessly.

1. Lightning App Builder – Fraud Detection App

- Created a new **Lightning App** named ***Fraud Detection***.
- Added navigation items: **Insurance Claims**, **Fraud Investigations**, **Reports**, and **Dashboards**.
- Created **Custom Tabs** for custom objects and added them to the app.
- **Used:**
 - To give project stakeholders a **dedicated workspace** to manage claims and fraud investigations without navigating multiple apps.
- **Steps Followed:**
 1. Setup → **App Manager** → **New Lightning App**.
 2. Enter *App Name*: Fraud Detection.
 3. Added navigation items (Insurance Claims, Fraud Investigations, Reports, Dashboards).
 4. Assigned the app to **Analyst, Manager, and Adjuster** profiles.

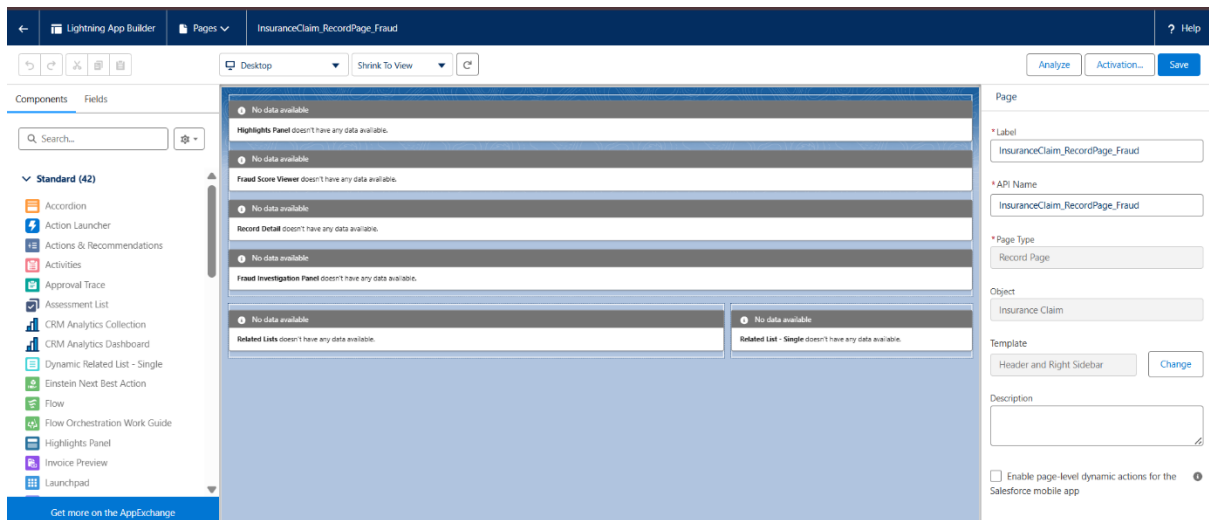


2. Record Pages – Claim Page Customization

- Customized the **Insurance Claim record page** with components:
 - *Highlights Panel* (Claim #, Status, Risk Score).
 - *Record Detail* (claim details).
 - *Related Lists* (documents, fraud investigations).
 - *Custom LWCs* → fraudScoreViewer, fraudInvestigationPanel.
- **Used:**
 - To provide a **single view of claim and fraud details** for faster decision-making.
- **Steps Followed:**
 1. Setup → **Lightning App Builder** → New Record Page.
 2. Selected **Insurance Claim** object.
 3. Dragged required components into layout.
 4. Added custom LWCs after deployment.
 5. Activated the page for the *Fraud Detection App*.

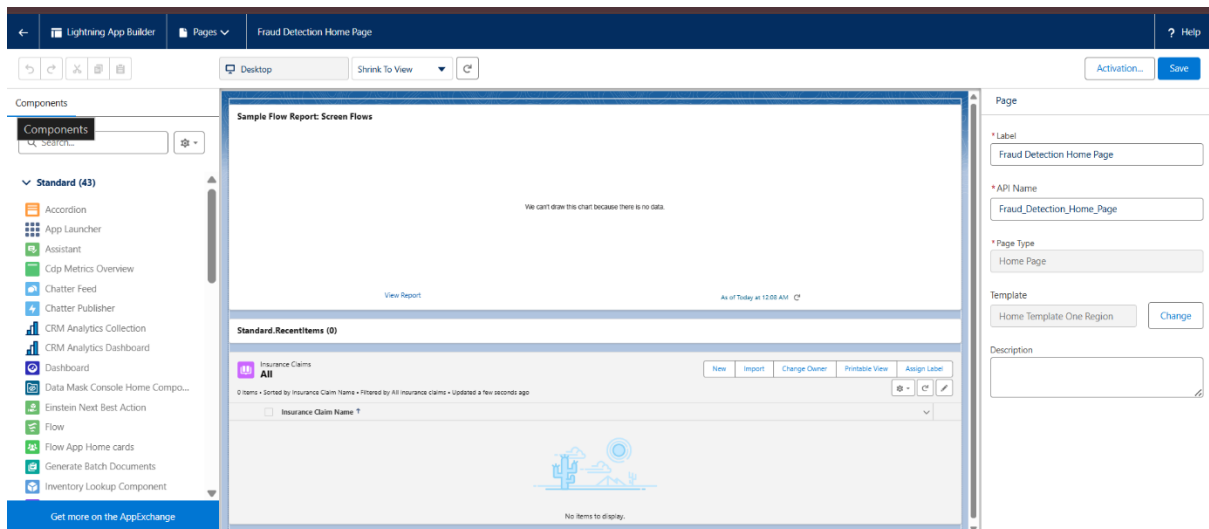
The screenshot displays the Lightning App Builder interface. At the top, the 'Lightning App Builder' header is visible. Below it, the 'Lightning Page' section shows the page name 'InsuranceClaim_RecordPage_Fraud' with 'Edit', 'Clone', and 'Delete' buttons. The 'Lightning Page Detail' section shows the page's information, including its name and label. Below this, the 'Assignments By App' section shows 'No Assignments to display'. The 'Assignments By App, Record Type, and Profile' section shows a table of assignments.

App	Record Type	Profile	Form Factor
Fraud Detection	Accident Claim	Claims Adjuster Profile	Desktop and phone
Fraud Detection	Medical Claim	Claims Adjuster Profile	Desktop and phone
Fraud Detection	Other Claim	Claims Adjuster Profile	Desktop and phone
Fraud Detection	Theft Claim	Claims Adjuster Profile	Desktop and phone
Fraud Detection	Accident Claim	Fraud Analyst Profile	Desktop and phone
Fraud Detection	Medical Claim	Fraud Analyst Profile	Desktop and phone
Fraud Detection	Other Claim	Fraud Analyst Profile	Desktop and phone
Fraud Detection	Theft Claim	Fraud Analyst Profile	Desktop and phone



3. Home Page Layouts:

- Created a Fraud Detection Home Page.
- Added components:
 - *Report Chart* → Fraud Trends.
 - *Recent Items*.
 - *List View* → High Risk Claims.
- **Used:**
 - To give Managers a dashboard-like overview of fraud risks and recent claims.
- **Steps Followed:**
 1. Setup → Lightning App Builder → New Home Page.
 2. Added components: *Report Chart*, *Recent Items*, *List View*.
 3. Activated for the *Fraud Detection App* and profiles.

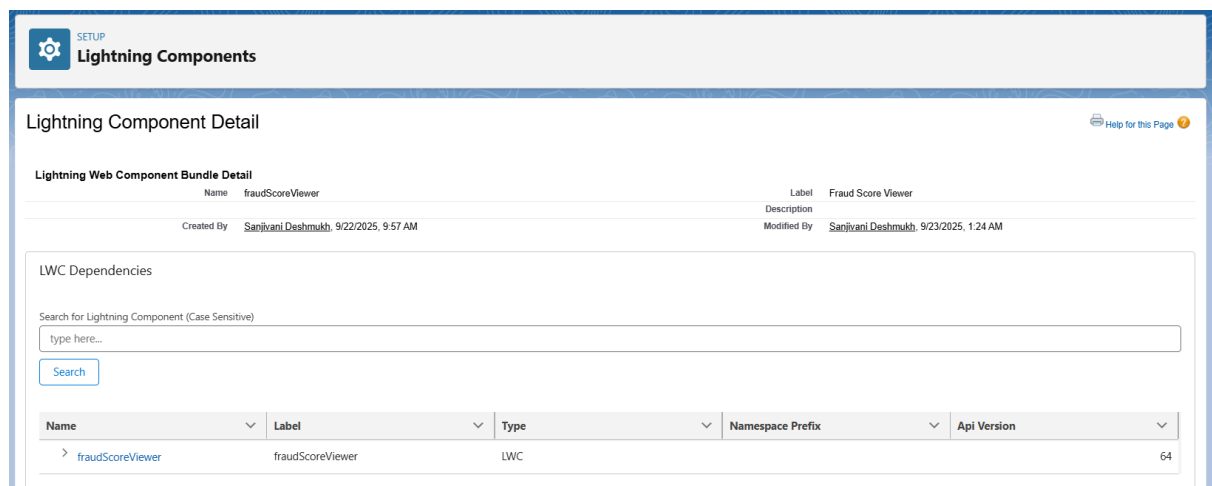


4. Lightning Web Components (LWCs)

Developed 3 LWCs to extend UI functionality:

(a) fraudScoreViewer

- **Purpose:** Displays real-time fraud score for a claim.
- **Usage:** Added to Insurance Claim Record Page.
- **Why:** Allows Analysts to instantly check fraud risk without running reports.



```
1 <template>
2   <lightning-card title="Claim Summary">
3     <div class="slds-m-around_medium">
4       <p><strong>Claim:</strong> {claimNumber}</p>
5       <p><strong>Status:</strong> {status}</p>
6       <p><strong>Amount:</strong> {amount}</p>
7       <p><strong>Risk Score:</strong> <lightning-badge label={riskScore}></lightning-badge>
8       <lightning-button label="Open Investigations" onclick={openInvestigations} class="slds-button--primary"></lightning-button>
9     </div>
10  </lightning-card>
11 </template>
12
```

```
1 import { LightningElement, api, wire } from 'lwc';
2 import { getRecord, getFieldValue } from 'lightning/uiRecordApi';
3 import CLAIM_NUMBER from '@salesforce/schema/Insurance_Claim__c.Claim_Number__c';
4 import CLAIM_AMOUNT from '@salesforce/schema/Insurance_Claim__c.Claim_Amount__c';
5 import RISK_SCORE from '@salesforce/schema/Insurance_Claim__c.Risk_Score__c';
6 import STATUS from '@salesforce/schema/Insurance_Claim__c.Status__c';
7
8 import { NavigationMixin } from 'lightning/navigation';
9
10 const FIELDS = [CLAIM_NUMBER, CLAIM_AMOUNT, RISK_SCORE, STATUS];
11
12 export default class FraudScoreViewer extends NavigationMixin(LightningElement) {
13   @api recordId;
14   @wire(getRecord, { recordId: '$recordId', fields: FIELDS })
15   claim;
16
17   get claimNumber() { return getFieldValue(this.claim.data, CLAIM_NUMBER) || '-'; }
18   get amount() { return getFieldValue(this.claim.data, CLAIM_AMOUNT) || '-'; }
19   get riskScore() { return getFieldValue(this.claim.data, RISK_SCORE) || '-'; }
20   get status() { return getFieldValue(this.claim.data, STATUS) || '-'; }
21
22   openInvestigations() {
23     this[NavigationMixin.Navigate]({
24       type: 'standard__recordRelationshipPage',
25       attributes: {
26         recordId: this.recordId,
27         relationshipApiName: 'Fraud_Investigations__r', // relationship name - verify in metadata
28         actionName: 'view'
29       }
30     });
31   }
32 }
```

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
3   <apiVersion>64.0</apiVersion>
4   <isExposed>true</isExposed>
5   <masterLabel>Fraud Score Viewer</masterLabel>
6   <targets>
7     <target>lightning__RecordPage</target>
8     <target>lightning__AppPage</target>
9   </targets>
10  <targetConfigs>
11    <targetConfig targets="lightning__RecordPage">
12      <property name="recordId" type="String" />
13    </targetConfig>
14  </targetConfigs>
15 </LightningComponentBundle>
16

```

(b) fraudInvestigationPanel

- **Purpose:** Shows related investigations for a claim, with option to create or update investigations.
- **Usage:** Added to record page layout.
- **Why:** Helps Analysts track fraud investigations directly from the claim page.

Lightning Component Detail

Lightning Web Component Bundle Detail

Name	Label
fraudInvestigationPanel	Fraud Investigation Panel

Created By: Sanjivani Deshmukh, 9/22/2025, 11:40 PM
Modified By: Sanjivani Deshmukh, 9/23/2025, 1:24 AM

LWC Dependencies

Search for Lightning Component (Case Sensitive)

type here...

Search

Name	Label	Type	Namespace Prefix	Api Version
fraudInvestigationPanel	fraudInvestigationPanel	LWC		64


```
EXPLORER
  FRAU...
  .sf
  .sfdx
  .vscode
  config
  force-app\main\def...
  applications
  aura
  classes
  contentassets
  flexipages
  layouts
  lwc
    fraudInvestigation...
      > _tests_
      fraudInvestigationPanel.html
      JS fraudInvestigation...
      fraudInvestigation...
    fraudScoreViewer
      > _tests_
      fraudScoreViewer...
      JS fraudScoreViewer...
      fraudScoreViewer...
  OUTLINE

force-app > main > default > lwc > fraudInvestigationPanel > fraudInvestigationPanel.html > template > lightning-card > template > t

1 <template>
2 <lightning-card title="Investigations">
3   <template if:true={invList}>
4     <template for:each={invList} for:item="inv">
5       <div key={inv.Id} class="slds-box slds-m-around_x-small">
6         <p><strong>ID:</strong> {inv.Name}</p>
7         <p><strong>Score:</strong> {inv.Fraud_Score__c}</p>
8         <p><strong>Status:</strong> {inv.Investigation_Status__c}</p>
9         <lightning-button variant="brand" label="Mark Closed" data-id={inv.Id} oncli
10       </div>
11     </template>
12   </template>
13   <template if:true={error}>
14     <div class="slds-text-color_error">{error}</div>
15   </template>
16 </lightning-card>
17 </template>
18
```

```
JS fraudInvestigationPanel.js
force-app > main > default > lwc > fraudInvestigationPanel > JS fraudInvestigationPanel.js > ...

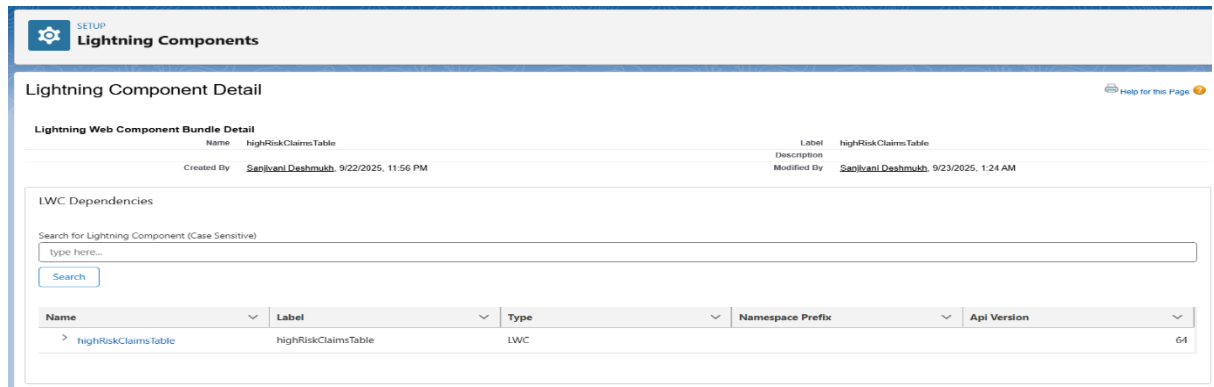
1 import { LightningElement, api, wire, track } from 'lwc';
2 import getInvestigationsByClaim from '@salesforce/apex/FraudLwcController.getInvestigationsByClaim';
3 import updateInvestigationStatus from '@salesforce/apex/FraudLwcController.updateInvestigationStatus';
4
5 export default class FraudInvestigationPanel extends LightningElement {
6   @api recordId;
7   @track invList;
8   @track error;
9
10   @wire(getInvestigationsByClaim, { claimId: '$recordId' })
11   wiredInvs({ error, data }) {
12     if (data) { this.invList = data; this.error = undefined; }
13     else if (error) { this.error = error.body ? error.body.message : error; this.invList = u
14   }
15
16   handleMarkClosed(event) {
17     const invId = event.target.dataset.id;
18     updateInvestigationStatus({ invId: invId, newStatus: 'Closed', notes: 'Closed via UI' })
19       .then(result => {
20         // dispatch event to parent to refresh
21         this.dispatchEvent(new CustomEvent('investigationupdated', { detail: { id: invId
22         return;
23       })
24       .catch(err => {
25         this.error = err.body ? err.body.message : err;
26       });
27   }
28 }
29
```

```
fraudInvestigationPanel.js-meta.xml
force-app > main > default > lwc > fraudInvestigationPanel > fraudInvestigationPanel.js-meta.xml > ...

1 <?xml version="1.0" encoding="UTF-8"?>
2 <LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
3   <apiVersion>64.0</apiVersion>
4   <isExposed>true</isExposed>
5   <masterLabel>Fraud Investigation Panel</masterLabel>
6   <targets>
7     <target>lightning__RecordPage</target>
8   </targets>
9 </LightningComponentBundle>
10
```

(c) highRiskClaimsTable

- **Purpose:** Displays all high-risk claims in a table view.
- **Usage:** Added to Home Page or Utility Bar.
- **Why:** Allows Managers to monitor all high-risk claims at a glance.



The screenshot shows the 'Lightning Component Detail' page in Salesforce. The component name is 'highRiskClaimsTable' and its label is 'highRiskClaimsTable'. It was created by 'Santvani Dashedmuh' on 9/22/2025 at 11:56 PM and modified on 9/23/2025 at 1:24 AM. The page includes a search bar for Lightning Components and a table listing the component's dependencies.

Name	Label	Type	Namespace Prefix	Api Version
highRiskClaimsTable	highRiskClaimsTable	LWC		64

```
<? highRiskClaimsTable.html >
force-app > main > default > lwc > highRiskClaimsTable > <? highRiskClaimsTable.html > ...
1  <template>
2    <lightning-card title="High Risk Claims">
3      <lightning-datatable
4        data={claims}
5        columns={columns}
6        key-field="Id"
7        onrowaction={handleRowAction}>
8      </lightning-datatable>
9    </lightning-card>
10  </template>
11
```

```
JS highRiskClaimsTable.js >
force-app > main > default > lwc > highRiskClaimsTable > JS highRiskClaimsTable.js > ...
1  import { LightningElement, track } from 'lwc';
2  import getHighRiskClaims from '@salesforce/apex/FraudLwcController.getHighRiskClaims';
3  import { NavigationMixin } from 'lightning/navigation';
4
5  const actions = [{ label: 'Open', name: 'open' }];
6  const cols = [
7    { label: 'Claim #', fieldName: 'Claim_Number__c' },
8    { label: 'Amount', fieldName: 'Claim_Amount__c', type: 'currency' },
9    { label: 'Risk', fieldName: 'Risk_Score__c', type: 'number' },
10   { type: 'action', typeAttributes: { rowActions: actions } }
11 ];
12
13 export default class HighRiskClaimsTable extends NavigationMixin(LightningElement) {
14   @track claims = [];
15   columns = cols;
16
17   connectedCallback() {
18     getHighRiskClaims({ minRisk: 70 })
19       .then(result => { this.claims = result; })
20       .catch(err => { console.error(err); });
21   }
22
23   handleRowAction(event) {
24     const actionName = event.detail.action.name;
25     const row = event.detail.row;
26     if (actionName === 'open') {
27       this[NavigationMixin.Navigate]({
28         type: 'standard__recordPage',
29         attributes: { recordId: row.Id, objectApiName: 'Insurance_Claim__c', actionName: 'view'
30       });
31     }
32   }
33 }
```

```
highRiskClaimsTable.js-meta.xml X
force-app > main > default > lwc > highRiskClaimsTable > highRiskClaimsTable.js-meta.xml > ...
1 <?xml version="1.0" encoding="UTF-8"?>
2 <LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
3   <apiVersion>64.0</apiVersion>
4   <isExposed>true</isExposed>
5   <targets>
6     <!-- Can be added on App Pages -->
7     <target>lightning__AppPage</target>
8
9     <!-- Can be added on Home Page -->
10    <target>lightning__HomePage</target>
11
12    <!-- Can be added on Record Pages -->
13    <target>lightning__RecordPage</target>
14
15    <!-- Can be added in Utility Bar -->
16    <target>lightning__UtilityBar</target>
17  </targets>
18 </LightningComponentBundle>
19
```

5. Apex with LWC Integration

- Created Apex Controllers (**FraudLwcController**) to fetch claim and investigation data.
- Granted Apex Class Access to Analyst and Manager profiles.
- **Used:**
 - LWC cannot directly access Salesforce objects beyond Lightning Data Service; Apex ensures complex queries and business logic can be handled.
- **Steps:**
 1. Setup → Apex Classes → Created controllers.
 2. Setup → Profiles → Added Apex Class Access.

The screenshot shows the Salesforce Setup interface for the 'FraudLwcController' Apex Class. The page includes a header with the 'Apex Classes' section and a sub-header 'Apex Class Detail'. Below this, there is a table with columns for Name, Namespace Prefix, Created By, Status, Code Coverage, and Last Modified By. The table shows the 'FraudLwcController' class, created by 'Sanjivani Deshmukh' on 9/22/2025, 9:06 AM, with a status of 'Active' and 0% code coverage. Below the table, there are tabs for 'Class Body', 'Class Summary', 'Version Settings', and 'Trace Flags'. The 'Class Body' tab is selected, showing the following Apex code:

```
1 public with sharing class FraudLwcController {
2   @AuraEnabled(cacheable=true)
3   public static List<Fraud_Investigation__c> getInvestigationsByClaimId(claimId) {
4     if (claimId == null) return new List<Fraud_Investigation__c>();
5     return [
6       SELECT Id, Name, Fraud_Score__c, Investigation_Status__c, Analyst_Notes__c
7       FROM Fraud_Investigation__c
8       WHERE Related_Claim__c = :claimId
9       ORDER BY CreatedDate DESC
10    ];
11  }
12
13  @AuraEnabled
14  public static Fraud_Investigation__c updateInvestigationStatus(Id invId, String newStatus, String notes) {
15    Fraud_Investigation__c inv = [SELECT Id, Investigation_Status__c, Analyst_Notes__c FROM Fraud_Investigation__c WHERE Id = :invId LIMIT 1];
16    inv.Investigation_Status__c = newStatus;
17    if (notes != null) inv.Analyst_Notes__c = notes;
18    update inv;
19    return inv;
20  }
21 }
```

6. Events in LWC

- Used Custom Events (**investigationupdated**) in **fraudInvestigationPanel** to notify parent components.
- **Used:**
 - Enables real-time refresh of data without reloading the page.

7. Wire Adapters & Imperative Calls

- **Wire Adapters:**
 - Used **@wire** with Apex methods to fetch claim and investigation data.
 - Why: Automatic refresh when underlying data changes.
- **Imperative Calls:**
 - Used for on-demand operations like creating investigations.
 - Why: Offers flexibility when updates should only happen after user action.

8. Navigation Service

- Used **NavigationMixin.Navigate** to navigate from **highRiskClaimsTable** to claim record page.
- **Used:**
 - Improves usability, allowing users to jump directly to claim details.

9. Permissions & Profiles

- Granted Apex Class Access and LWC access to Analyst and Manager profiles.
 - Mapped Lightning Record Pages and Home Page to Fraud Detection App.
 - **Used:**
 - To ensure only authorized users (Analyst/Manager) can access fraud-related functionality.
-

Phase 7: Integration & External Access

Goal:

Enable Salesforce to securely communicate with external systems (fraud detection API, other services) while ensuring data security, scalability, and compliance with Salesforce governor limits.

1. Named Credentials (✓ Used)

Purpose:

Securely store the base URL and authentication settings for an external API without hardcoding credentials inside Apex classes.

Steps Followed:

1. Setup → Named Credentials → New.
2. Added **Label: FraudAPI** and **URL: https://fraudapi.example.com**.
3. Chose **Identity Type: Anonymous** and **Authentication Protocol: No Authentication**, since using a demo API without login credentials.
4. Saved the configuration.

Why Important?

- Avoids hardcoding API endpoints in Apex.
- Centralized management of authentication.
- Secure and easy to update if endpoint changes.

The screenshot shows the Salesforce Setup interface for configuring a Named Credential. The header includes the 'SETUP' icon and the title 'Named Credentials'. The main heading is 'Named Credential: FraudDetectionAPI'. Below this, a sub-header states: 'Specify the callout endpoint's URL and the authentication settings that are required for Salesforce to make callouts to the remote system.' A link 'Back to Named Credentials' is provided. The configuration form includes fields for 'Label' (FraudDetectionAPI), 'Name' (FraudDetectionAPI), and 'URL' (https://fraudapi.example.com). There are 'Edit' and 'Delete' buttons. The 'Authentication' section is expanded, showing 'Certificate' as the selected identity type, 'Named Principal' as the identity type, and 'No Authentication' as the authentication protocol. The 'Callout Options' section is also expanded, showing 'Generate Authorization Header' checked, 'Allow Merge Fields in HTTP Header' unchecked, 'Allow Merge Fields in HTTP Body' unchecked, and 'Outbound Network Connection' unchecked.

Field	Value
Label	FraudDetectionAPI
Name	FraudDetectionAPI
URL	https://fraudapi.example.com

Authentication

Field	Value
Identity Type	Named Principal
Authentication Protocol	No Authentication

Callout Options

Field	Value
Generate Authorization Header	☒
Allow Merge Fields in HTTP Header	☐
Allow Merge Fields in HTTP Body	☐
Outbound Network Connection	☐

2. External Services (❌ Not Used)

Purpose:

Allows Salesforce to consume APIs described by an OpenAPI (Swagger) specification without writing code. Salesforce auto-generates Apex actions from the service schema.

Reason for Skipping:

- Our demo fraud detection API doesn't provide Swagger/OpenAPI definitions.
- Instead, we directly integrated via Named Credentials + Apex Callout.

3. Web Services (REST/SOAP) (✔ Used – REST)

Purpose:

Salesforce supports integrating with external systems via REST or SOAP APIs.

- REST → lightweight, JSON-based (used in our project).
- SOAP → XML-based, older format (not used here).

Implementation:

- Created an Apex class with `@future(callout=true)` to send REST callouts.
- Example: Fetch fraud score for a claim using `req.setEndpoint('callout:FraudAPI/score')`.

Code:

```
public with sharing class FraudAPIService {

    // Future method for async callout
    @future(callout=true)
    public static void updateFraudScore(Id claimId) {
        try {
            // Get claim record
            Insurance_Claim__c claim = [
                SELECT Id, Claim_Number__c
                FROM Insurance_Claim__c
                WHERE Id = :claimId
                LIMIT 1
            ];
```

```

Http http = new Http();
HttpRequest req = new HttpRequest();

// Use Named Credential → FraudAPI_NC (created earlier)
req.setEndpoint('callout:FraudAPI_NC/checkFraud?claimId=' + claim.Claim_Number_
_c);
req.setMethod('GET');

HttpResponse res = http.send(req);

if (res.getStatusCode() == 200) {
    // Parse JSON response
    Map<String, Object> result =
        (Map<String, Object>) JSON.deserializeUntyped(res.getBody());

    if (result.containsKey('fraudScore')) {
        Decimal score = Decimal.valueOf(result.get('fraudScore').toString());

        // Update claim with fraud score
        claim.Risk_Score__c = score;
        update claim;
    }
} else {
    System.debug('Fraud API Callout failed → ' + res.getStatusCode() + ' : ' + res.getBo
y());
}
} catch (Exception e) {
    System.debug('Error in FraudAPIService.updateFraudScore: ' + e.getMessage());
}
}
}

```


Apex Classes

Apex Class

FraudAPIService

Help for

Apex Class Detail

Edit

Delete

Download

Security

Show Dependencies

Name	FraudAPIService	Status	Active
Namespace Prefix		Code Coverage	0% (0/17)
Created By	Sanjivani Dashmukh , 9/23/2025, 6:20 AM	Last Modified By	Sanjivani Dashmukh , 9/23/2025, 6:38 AM

Class Body

Class Summary

Version Settings

Trace Flags

```

1 public with sharing class FraudAPIService {
2
3     // Future method for async callout
4     @future(callout=true)
5     public static void updateFraudScore(Id claimId) {
6         try {
7             // Get claim record
8             Insurance_Claim__c claim = [
9                 SELECT Id, Claim_Number__c
10                FROM Insurance_Claim__c
11                WHERE Id = :claimId
12                LIMIT 1
13            ];
14
15            Http http = new Http();
16            HttpRequest req = new HttpRequest();
17
18            // Use Named Credential → FraudAPI_NC (created earlier)
19            req.setEndpoint('callout:FraudAPI_NC/checkFraud?claimId=' + claim.Claim_Number__c);
20            req.setMethod('GET');
21
22            HttpResponse res = http.send(req);

```

4. Callouts (✔ Used)

Purpose:

Allow Salesforce to send HTTP requests (GET/POST) to external systems.

Steps:

1. Added Named Credential → FraudAPI.
2. Wrote Apex code:
3. `HttpRequest req = new HttpRequest();`
4. `req.setEndpoint('callout:FraudAPI/score');`
5. `req.setMethod('GET');`
6. `HttpResponse res = new Http().send(req);`
7. Parsed JSON response and updated Claim Risk Score.

Why Important?

- Enables Salesforce to interact with real-time fraud detection services.
- Provides fraud score enrichment for Insurance Claims.

SETUP

Remote Site Settings

Remote Site Details

Remote Site Detail

Edit

Delete

Clone

Remote Site Name

FraudAPI

Modified By

Sanjivani Deshmukh

9/23/2025, 6:33 AM

Remote Site URL

https://fraudapi.example.com

Disable Protocol Security

☐

Description

Active

☒

Created By

Sanjivani Deshmukh

9/23/2025, 6:33 AM

Edit

Delete

Clone

5. Platform Events (Used)

Purpose:

Enable event-driven architecture for real-time communication inside Salesforce and with external systems.

Example in Our Project:

- Created a Fraud_Alert__e platform event.
- Published event when claim fraud score > 80.
- External listeners can react to fraud alerts immediately.

Why Important?

- Decouples fraud detection logic from notifications.
- Supports real-time alerting and system integration.

SETUP

Platform Events

Platform Event

FraudAlertEvent

Standard Fields (4)

Custom Fields & Relationships (4)

Platform Event Definition Detail

Edit

Delete

Singular Label

FraudAlertEvent

Description

Plural Label

FraudAlertEvents

Deployment Status

Deployed

Object Name

FraudAlertEvent

API Name

FraudAlertEvent__e

Event Type

High Volume

i

Publish Behavior

Publish Immediately

i

Created By

Sanjivani Deshmukh

9/23/2025, 6:46 AM

Modified By

Sanjivani Deshmukh

9/23/2025, 6:46 AM

Standard Fields

Action	Field Label	Field Name	Data Type	Controlling Field	Indexed
Created By		CreatedBy	Lookup(User)		
Created Date		CreatedDate	Date/Time		
Event UUID		EventId	Text(36)		
Replay ID		ReplayId	External Lookup		

Custom Fields & Relationships

New

Action	Field Label	API Name	Data Type	Indexed	Controlling Field	Modified By		
Field	Detail	ClaimNumber	c	ClaimNumber	c	Text(20)	Sanjivani Deshmukh	9/23/2025, 6:47 AM

```
File Edit Debug Test Workspace Help < >
FraudAlertPublisher.apxc
Code Coverage: None API Version: 64
1 public class FraudAlertPublisher {
2
3     // Method to publish the Fraud Alert Event
4     public static void publishFraudAlert(String claimNumber, Decimal fraudScore, String customerName) {
5
6         // Create an instance of your Platform Event
7         FraudAlertEvent__e alert = new FraudAlertEvent__e(
8             ClaimNumber_c__c = claimNumber,
9             FraudScore_c__c = fraudScore,
10            CustomerName_c__c = customerName,
11            Message_c__c = 'High-risk claim detected'
12        );
13
14        // Publish the event
15        Database.SaveResult result = EventBus.publish(alert);
16
17        if (result.isSuccess()) {
18            System.debug('Fraud Alert Event Published Successfully: ' + claimNumber);
19        } else {
20            System.debug('Failed to publish event: ' + result.getErrors()[0].getMessage());
21        }
22    }
23 }
```

 **Apex Triggers**

Apex Trigger

FraudAlertEventTrigger

Apex Trigger Detail

Edit Delete Download Show Dependencies

Name	FraudAlertEventTrigger	sObject Type	FraudAlertEvent
Code Coverage	0% (0/1)	Status	Active
Created By	Sanjivani Deshmukh, 9/23/2025, 7:07 AM	Last Modified By	Sanjivani Deshmukh, 9/23/2025, 7:13 AM
Namespace Prefix			

Apex Trigger

Version Settings Trace Flags

```
1 trigger FraudAlertEventTrigger on FraudAlertEvent__e (after insert) {
2     for(FraudAlertEvent__e eventRecord : Trigger.New) {
3         System.debug('Fraud Alert for Claim: ' + eventRecord.ClaimNumber_c__c);
4         // Call external system via Apex callout if needed
5     }
6 }
```

Edit Delete Download Show Dependencies

6. Change Data Capture (❌ Not Used)

Purpose:

Automatically publishes events whenever records in selected objects change.

Reason for Skipping:

- Not essential for our demo project.
 - Could be used in future for syncing Claim/Investigation updates with external databases.
-

7. Salesforce Connect (❌ Not Used)

Purpose:

Connects external databases (like Oracle, SAP, AWS) as virtual objects in Salesforce without storing data locally.

Reason for Skipping:

- Our fraud detection service provides API access only, not database connection.
 - Salesforce Connect not required.
-

8. API Limits

Purpose:

Salesforce imposes limits to ensure system performance.

Important Limits:

- **REST API Calls per 24 hours** → Based on license (e.g., 15,000 for Enterprise Edition).
- **Callout timeout** → 120 seconds max.
- **Concurrent callouts** → Up to 10 per Apex transaction.

Why Important?

- Helps us design integrations without exceeding limits.
 - In real projects, ensures API quota is not exhausted.
-

9. OAuth & Authentication (❌ Not Used – Chose No Auth)

Purpose:

OAuth2 is a standard authentication method for connecting Salesforce to third-party services.

Why Skipped:

- Our fraud detection API is a demo and doesn't require credentials.
- Chose **No Authentication** in Named Credentials.

Note:

In a real system, APIs usually require OAuth2 / Password Authentication for security.

10. Remote Site Settings (✔ Used)

Purpose:

Allow Salesforce to call external services securely.

Steps:

- For Named Credentials → Remote Site Settings are automatically handled.
- If calling endpoint directly, you must manually add base URL in **Setup → Remote Site Settings**.

Why Important?

- Prevents unauthorized external callouts.
- Adds a layer of security by whitelisting allowed domains.

The screenshot shows the 'Remote Site Settings' page in Salesforce. At the top, there is a 'SETUP' icon and the title 'Remote Site Settings'. Below this, the 'Remote Site Details' section is visible. It contains a table with the following information:

Remote Site Detail		Edit	Delete	Clone
Remote Site Name	FraudAPI			
Remote Site URL	https://fraudapi.example.com			
Disable Protocol Security	☐			
Description				
Active	☒			
Created By	Sanjivani Deshmukh, 9/23/2025, 6:33 AM	Edit	Delete	Clone

Phase 8: Data Management & Deployment

 **Goal:** Manage financial transaction data efficiently and move project configurations between environments.

1. Data Import Wizard:

Purpose: Import small demo datasets into Salesforce for testing.

Steps Followed:

- Created a sample CSV with ~50 **demo financial transactions** (Transaction ID, Customer ID, Amount, Location, Risk Score, Fraud Flag).
- Used **Data Import Wizard** (Setup → Data Import Wizard → Custom Objects → Transaction).
- Imported the records successfully (Claim/Transaction ID auto-generated if using Auto Number).

Why Important?

Provides test data for fraud monitoring flows, validations, and dashboards.

 **Bulk Data Load Jobs**

Monitor Bulk Data Load Jobs [Help for this Page](#)

Monitor the status of recent bulk data load jobs. These jobs are created by Data Loader and other Bulk API client applications.

Quota

Your organization has processed 0 batches in the last 24 hours. Your organization can process 15,000 batches in a 24-hour period.

Resource used in the last 24 hours:
CPU: 0 milliseconds
IO: 0 bytes
Disk: 0 bytes

In Progress

Job ID	Submitted By	Start Time	Status	Job Type	Operation	Object	Records Processed	Records Failed	Progress
No records to display.									

Completed last 7 days

Job ID	Submitted By	Start Time	End Time	Status	Job Type	Operation	Object	Records Processed	Records Failed	Time to Complete (hh:mm:ss)
750gk000000Dhvpv	Deshmukh_Sanjivani	9/24/2025, 5:56 AM	9/24/2025, 5:56 AM	Closed	Bulk V1	Insert	Insurance Claim	50	0	00:01

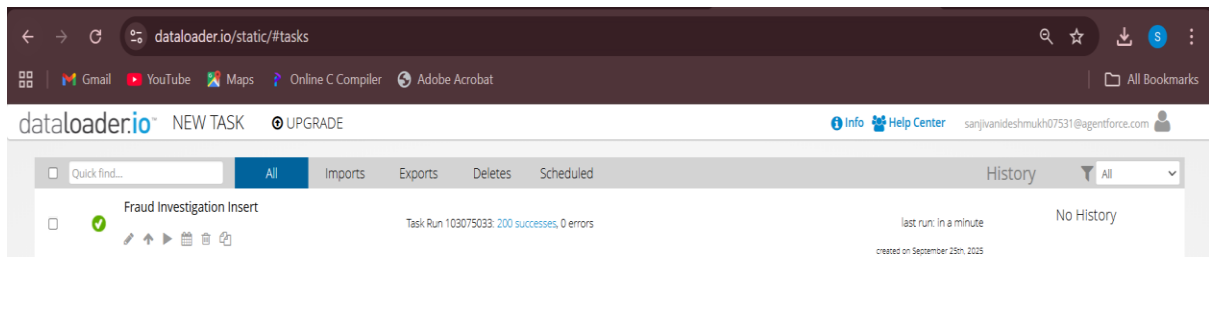
2. Data Loader (Bulk Data):

Purpose: Import or export large datasets (50k+ records).

- Can be used if you want to test system performance with **bulk financial transactions**.
- Supports insert, update, upsert, delete.

Why Important?

Useful for **stress testing fraud detection algorithms** with bigger data.



3. Duplicate Rules:

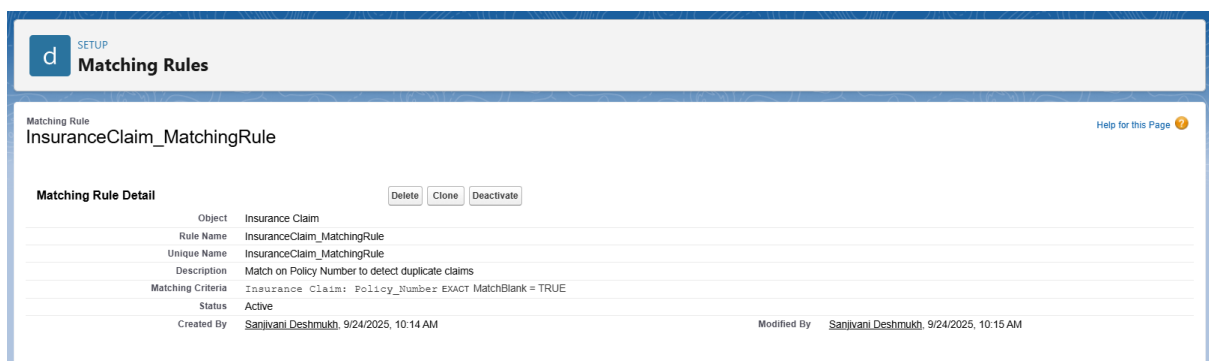
Purpose: Prevent duplicate **transaction records** or **customer profiles**.

Steps:

- Setup → Duplicate Rules → New Rule.
- For **Customer Profile**, match on Email/Phone.
- For **Transactions**, match on Transaction ID.

Why Important?

Fraud detection depends on accurate data — duplicates could create false alerts.



SETUP

Duplicate Rules

Insurance Claim Duplicate Rule

InsuranceClaim_DuplicateRule

Help for this Page

Duplicate Rule Detail

Edit

Delete

Clone

Deactivate

Rule Name	InsuranceClaim_DuplicateRule	Order	1 of 1	Reorder
Description	Prevents duplicate Insurance Claims based on Policy Number.			
Object	Insurance Claim			
Record-Level Security	Enforce sharing rules			
Action On Create	Allow	Operations On Create	☒ Alert	☐ Report
Action On Edit	Allow	Operations On Edit	☒ Alert	☐ Report
Alert Text	Use one of these records?			
Active	☒			
Matching Rule	☒ InsuranceClaim_MatchingRule	☒ Mapped	Matching Criteria	
Conditions	Insurance Claim: Policy_Number EXACT MatchBlank = TRUE			
Created By	Sanjivani Deshmukh 9/24/2025, 10:04 AM		Modified By	
			Sanjivani Deshmukh 9/24/2025, 10:16 AM	

Edit

Delete

Clone

Deactivate

4. Data Export & Backup:

Purpose: Ensure regular backup of fraud data.

Steps:

- Setup → Data Export → Schedule Weekly Export.
- Export all custom objects: Transaction, Customer Profile, Fraud Investigation.
- Store CSV backups securely.

Why Important?

Prevents data loss during errors or deployment, ensures audit history of fraud investigations.

SETUP

Data Export

Monthly Export Service

Help for this Page

Data Export lets you prepare a copy of all your data in salesforce.com. From this page you can start the export process manually or schedule it to run automatically. When an export is ready for download you will receive an email containing a link that allows you to download the file(s). The export files are also available on this page for 48 hours, after which time they are deleted.

Next scheduled export: 10/1/2025, 12:00 AM

Export Now

Schedule Export

5. Change Sets:

Purpose:

Change Sets are normally used to move custom objects, fields, Apex classes, flows, and Lightning pages from Sandbox to Production without re-building.

Steps (real-world process):

1. In Sandbox Org → Setup → Outbound Change Sets → New Change Set.
2. Add components: Custom Objects, Apex Classes, Flows, Validation Rules, Approval Processes, Lightning Pages.
3. Upload Change Set to Target Org.
4. In Production Org → Inbound Change Sets → Deploy.

Note for Our Project:

Since we are working in a Developer Edition Org, Change Sets are not available.

Instead, we documented the process for completeness but used VS Code & SFDX to manage deployments in our project.

6. Unmanaged vs Managed Packages (Optional)

- **Unmanaged:** Share project with classmates → editable.
- **Managed:** Locked → AppExchange style.

Our Case: Use Unmanaged Package if sharing project. No AppExchange publishing needed.

7. ANT Migration Tool:

Purpose: Command-line deployment tool.

- Used in enterprise teams with CI/CD pipelines.
 - Not required for this project.
-

8. VS Code & SFDX:

Purpose:

VS Code with Salesforce CLI (SFDX) gives a developer-friendly way to manage Salesforce metadata, automate deployments, and maintain version control with Git. Unlike Change Sets, this approach works in Developer Orgs and is preferred for real projects.

Steps Followed:**1. Install Salesforce CLI (SFDX):**

- Download from Salesforce CLI website.
- Install it on your system.

2. Install VS Code & Salesforce Extension Pack:

- Install Visual Studio Code.
- In VS Code → Extensions Marketplace → Search Salesforce Extension Pack → Install.

3. Authorize Dev Org in VS Code:

- Open Command Palette in VS Code (Ctrl + Shift + P).
- Type: SFDX: Authorize an Org.
- Choose “Dev Hub” or “Sandbox/Developer Org” → Log in via browser → Allow access.

4. Create/Connect Project:

- In VS Code → Command Palette → SFDX: Create Project with Manifest.
- Select a folder for your project.
- This generates a manifest/package.xml file.

5. Pull Metadata from Org:

- In the terminal (inside your VS Code project folder), run:

`sf project retrieve start --manifest manifest/package.xml`

- This fetches all metadata defined in package.xml (e.g., custom objects, Apex classes, LWCs).

The screenshot shows a VS Code window with a terminal running the command `sf project retrieve start --manifest manifest/package.xml`. The terminal output indicates a successful retrieval of metadata from the Salesforce org. A table titled "Retrieved Source" lists the retrieved items.

```
PS C:\Users\91899\OneDrive\Desktop\Git clone\FraudDetectionLWC> sf project retrieve start --manifest manifest/package.xml
>>
» Warning: @salesforce/cli update available from 2.104.6 to 2.107.6.

- [4/4] Done 0ms

Status: Succeeded

Retrieving Metadata

Retrieving v64.0 metadata from sanjivanideshmukh07531@agentforce.com using the v64.0 SOAP API
✓ Preparing retrieve request 52ms
✓ Sending request to org 608ms
✓ Waiting for the org to respond 2.42s
✓ Done 0ms

Status: Succeeded
Elapsed Time: 3.03s
```

State	Name	Type	Path
Created	ClaimRiskRecalcBatch	ApexClass	force-app\main\default\classes\ClaimRiskRecalcBatch.cls
Created	ClaimRiskRecalcBatch	ApexClass	force-app\main\default\classes\ClaimRiskRecalcBatch.cls-meta.xml
Created	ClaimRiskRecalcScheduler	ApexClass	force-app\main\default\classes\ClaimRiskRecalcScheduler.cls
Created	ClaimRiskRecalcScheduler	ApexClass	force-app\main\default\classes\ClaimRiskRecalcScheduler.cls-meta.xml

Phase 9: Reporting, Dashboards & Security Review

Goal:

Monitor insurance claims, detect high-risk/fraudulent claims, track investigation progress, analyze trends, and secure sensitive customer and fraud data.

1. Reports

Step 1: Go to Reports

1. In Salesforce, click the **App Launcher** (⚡).
 2. Search for **Reports** → Click **New Report**.
-

Step 2: Select Report Type

1. Choose the **Insurance Claim** report type.
 - If you have a **custom report type** (Claims + Fraud Investigations), you can use that too.
 2. Click **Continue**.
-

Step 3: Add Filters

1. **Date Range:** Set to **All Time** or a relevant range (e.g., **Last 90 Days**).
2. **Fraud Score:** Keep all values for now (we'll calculate averages).
3. **Claim Status:** Optionally filter for **Approved** or **Closed** claims if you only want finalized ones.

Step 4: Add Columns

Make sure the following fields are included:

- **Claim Number**
 - **Claim Type** (e.g., Medical, Vehicle, Property, etc.)
 - **Claim Amount**
 - **Fraud Score**
 - **Fraud Flag (High Risk Yes/No)** → This is usually derived if **Fraud Score > 80**.
-

Step 5: Group by Claim Type

1. In the report builder, **drag Claim Type** into the **Group Rows** section.
 - This will organize all claims under their type.
-

Step 6: Summarize Fraud Score

1. Hover over **Fraud Score** column → Click ▼ → **Summarize**.
2. Choose **Average**.
 - This will show the average fraud score per claim type.

Step 7: Save the Report

1. Click **Save & Run**.
2. Report Name: **Claim Type vs Fraud Score Report**.
3. Folder: **Fraud Monitoring Reports**.

Fraud Detection Insurance Claims High Risk Claims Fraud Investigations Reports Dashboards				
Report: Insurance Claims Claim Type vs Risk Score Report Enable Field Editing Q Add Chart ▼ ↻ Edit				
Total Records Total Claim Amount Total Risk Score Average Risk Score 45 ₹46,80,972 560 12				
Claim Type Insurance Claim: Insurance Claim Name Claim Amount Claim Number Risk Score				
Accident (8) Subtotal	a02gK000005I2GI	₹59,353	CLM-0053	10
	a02gK000005I2Gs	₹84,103	CLM-0060	10
	a02gK000005I2H6	₹31,877	CLM-0074	0
	a02gK000005I2HE	₹1,45,650	CLM-0082	10
	a02gK000005I3uB	₹59,353	CLM-0019	10
	a02gK000005I3uG	₹84,103	CLM-0024	10
	a02gK000005I3uN	₹31,877	CLM-0031	0
	a02gK000005I3uR	₹1,45,650	CLM-0035	10
		₹6,41,966		60 Avg: 8
Theft (8)	a02gK000005I2Ga	₹1,71,447	CLM-0042	25
	a02gK000005I2Gg	₹9,327	CLM-0048	15
	a02gK000005I2GL	₹60,490	CLM-0050	25
Row Counts Detail Rows Subtotals Grand Total ✓ ✓ ✓ ✓				

2. Custom Report Types

- Object:** Insurance Claim + Fraud Investigation.
- Fields:** Claim Number, Customer Name, Claim Amount, Fraud Score, Investigation Status, Analyst Notes.
- Summary Reports:** Aggregate Fraud Scores by Claim Type, Region, or Analyst.
- Purpose:** Enable detailed reporting and dashboards to analyze high-risk claims, trends, and analyst performance.

Report: Claims with Fraud Investigations

Fraud Detection Summary

Enable Field Editing

Q

Add Chart

▼

↺

Edit

▼

Total Records

9

Total Claim Amount

₹6,92,908

Total Fraud Score

545

Average Fraud Score

61

Claim Type	Insurance Claim Name	Fraud Investigation Name	Claim Amount	Claim Number	Fraud Score	Analyst Notes	Status
Accident (2)	a02gK000005I09n	Investigation_100	₹1,72,890	CLM-0003	64	Escalated to senior analyst	In Review
	a02gK000005I09n	Investigation_100	₹1,72,890	CLM-0003	64	Escalated to senior analyst	In Review
	Subtotal		₹1,72,890		128 Avg: 64		
Theft (1)	a02gK000005I2Gb	Investigation_107	₹5,923	CLM-0043	54	Suspicious activity detected	In Review
Subtotal			₹5,923		54 Avg: 54		
Medical (2)	a02gK000005I2Gd	Investigation_106	₹17,266	CLM-0045	55	Needs further review	Approved
	a02gK000005I09s	Investigation_109	₹94,870	CLM-0008	69	Needs further review	In Review
	Subtotal			₹1,12,136		124 Avg: 62	
Other (4)	a02gK000005I2H5	Investigation_103	₹1,59,201	CLM-0073	65	Case resolved with no fraud	Submitted
	a02gK000005I2Gc	Investigation_107	₹1,09,093	CLM-0044	54	Suspicious activity detected	Approved
	a02gK000005I2H0	Investigation_103	₹1,33,665	CLM-0068	65	Case resolved with no fraud	In Review
	Row Counts		Detail Rows Subtotals Grand Total				

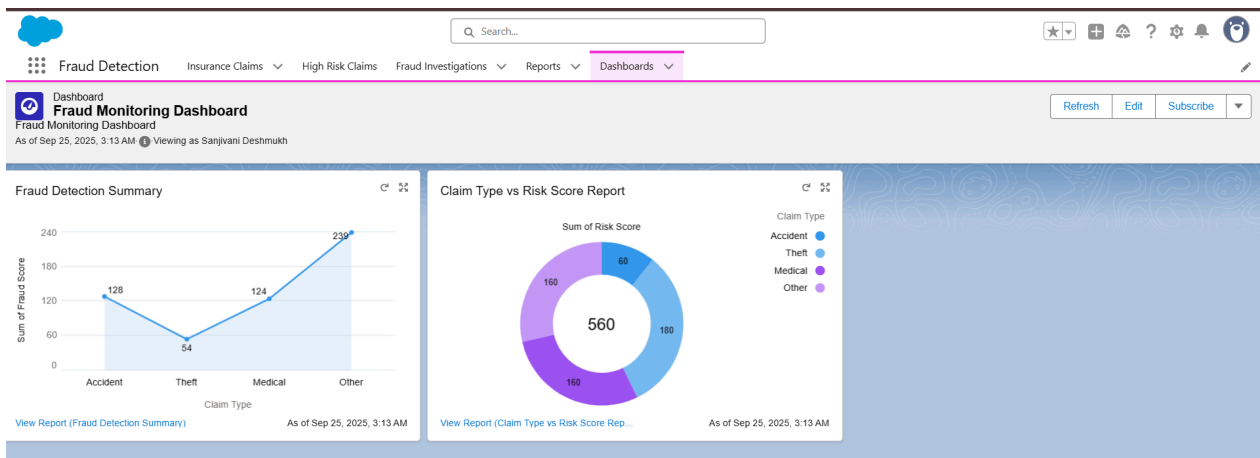
3. Dashboards

Visualize fraud patterns and business impact:

- **Fraud Monitoring Dashboard:** Donut chart showing Claim Type vs Risk Score Report and Line chart showing Fraud Detection Summary.

Steps:

1. Go to **Dashboards** → **New Dashboard**.
2. Add components using custom reports.
3. Set chart types and groupings.
4. Apply filters (Time Frame, Claim Type, Analyst).
5. Save and refresh to display live data.



4. Sharing Settings

- **Sharing Settings:** Claims private, Fraud Investigations inherit claim visibility.

Customer Profile	Public Read Only	Private	✓
Fraud Investigation	Private	Private	✓
Insurance Claim	Private	Private	✓

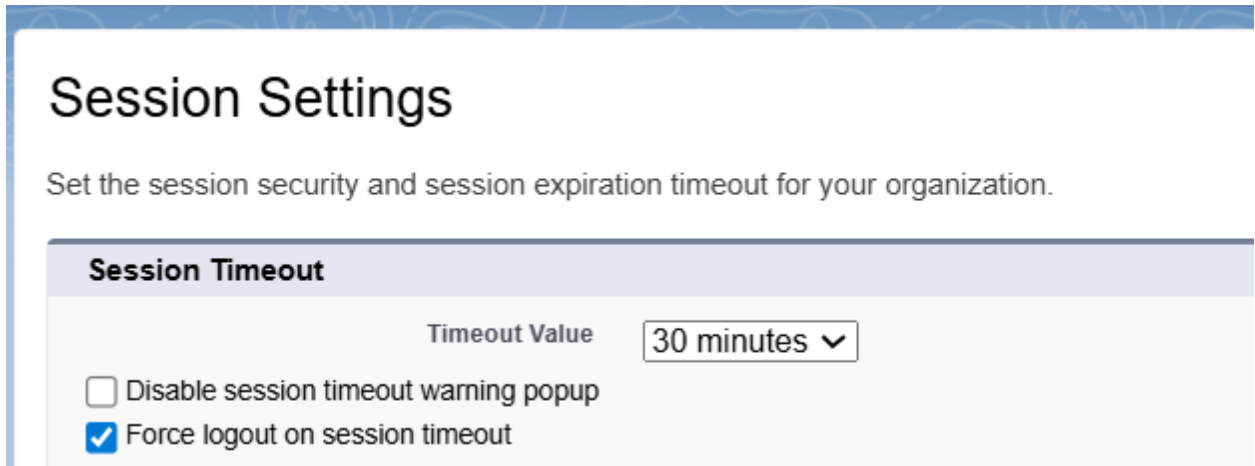
5. Field Level Security

- Hide sensitive fields like Customer ID Proof and Bank Account Details from junior analysts.
- Fraud Score visible only to analysts and managers.

Steps: Setup → Object Manager → Field-Level Security → adjust visibility per profile.

6. Session Settings

- **Session Timeout:** 30 minutes of inactivity → auto logout.



Session Settings

Set the session security and session expiration timeout for your organization.

Session Timeout

Timeout Value: 30 minutes ▼

☐ Disable session timeout warning popup

☒ Force logout on session timeout

7. Login IP Ranges

Login IP Ranges: Restrict access to office VPN or secure networks.

Steps: Setup → Profiles (Claims Adjuster Profile) → Session Settings & Login IP Ranges.



Login IP Ranges

Restrict access to office VPN or secure networks.

Login IP Ranges

New

Login IP Ranges Help ?

Action	IP Start Address	IP End Address	Description
Edit Del	103.111.133.1	103.111.133.255	

8. Audit Trail

- Track who created, modified, or deleted claims and investigations.
- Enables **compliance and accountability**.
- Steps: Setup → Audit Trail / Field History Tracking → enable for key objects.



SETUP

View Setup Audit Trail

View Setup Audit Trail

[Help for this Page](#)

The last 20 entries for your organization are listed below. You can [download](#) your organization's setup audit trail for the last six months (Excel .csv file).

View Setup Audit Trail

Date	User	Source Namespace Prefix	Action	Section	Delegate User ?
9/25/2025, 3:38:45 AM PDT	sanjivanideshmukh07531@agentforce.com		Changed Session Timeout Value from 120 to 30 minutes	Session Settings	
9/24/2025, 9:21:54 PM PDT	sanjivanideshmukh07531@agentforce.com		Changed highRiskClaimsTable Lightning Web Component	Lightning Components	
9/24/2025, 9:21:54 PM PDT	sanjivanideshmukh07531@agentforce.com		Changed fraudScoreViewer Lightning Web Component	Lightning Components	
9/24/2025, 9:21:54 PM PDT	sanjivanideshmukh07531@agentforce.com		Changed fraudInvestigationPanel Lightning Web Component	Lightning Components	
9/24/2025, 10:16:16 AM PDT	sanjivanideshmukh07531@agentforce.com		For the 01lgK000002IHqr duplicate rule InsuranceClaim_DuplicateRule, changed "Active" from "false" to "true"	Duplicate Rule	
9/24/2025, 10:15:54 AM PDT	sanjivanideshmukh07531@agentforce.com		Insurance Claim matching rule, InsuranceClaim_MatchingRule, activating by Sanjivani Deshmukh	Matching Rule	
9/24/2025, 10:15:32 AM PDT	sanjivanideshmukh07531@agentforce.com		For duplicate rule InsuranceClaim_DuplicateRule, changed matching rules.	Duplicate Rule	
9/24/2025, 10:14:28 AM PDT	sanjivanideshmukh07531@agentforce.com		For matching rule InsuranceClaim_MatchingRule, added matching criteria where matching method is Exact, the field is Policy_Number and match blank fields is "Match When Both Blank"	Matching Rule	
9/24/2025, 10:14:28 AM PDT	sanjivanideshmukh07531@agentforce.com		For matching rule InsuranceClaim_MatchingRule, matching engine set to Exact Match Engine.	Matching Rule	
9/24/2025, 10:14:28 AM PDT	sanjivanideshmukh07531@agentforce.com		For matching rule InsuranceClaim_MatchingRule, changed Advanced Logic from null to 1	Matching Rule	